A

PROJECT REPORT

ON

"GREENVISIONAI: An AI-Based Framework for Detecting Illegal Mining and Deforestation Using Sentinel-2 Satellite Imagery"

SUBMITTED TO

SAVITRIBAI PHULE PUNE UNIVERSITY
FOR THE PARTIAL FULFILLMENT OF THE DEGREE OF

BACHELOR OF ENGINEERING

SUBMITTED BY

MS. BARKARE VEDIKA NANDKISHOR	**EXAM NO. : B401140084**
MS. PATIL PRANJALI MOHAN	**EXAM NO. : B401140112**
MR. PATIL SHUBHAM DNYANESHWAR	**EXAM NO. : B401140114**
MR. SONAWANE PRAVIN RAJESH	**EXAM NO. : B401140124**

UNDER GUIDANCE OF

MS. S. P. BAVISKAR

ISO 21001:2018 Certified | NAAC Accredited

Jawahar Education Society's,
INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NASHIK.
(Approved by AICTE, New Delhi, DTE, Government of Maharashtra, Affiliated to Savitribai Phule Pune University)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE (AI) AND DATA SCIENCE
JAWAHAR EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NASHIK.

ACADEMIC YEAR 2025-2026

ISO 21001:2018 Certified | NAAC Accredited

Jawahar Education Society's,
INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NASHIK.
(Approved by AICTE, New Delhi, DTE, Government of Maharashtra, Affiliated to Savitribai Phule Pune University)

CERTIFICATE

This is to certify that the project report entitled

"GREENVISIONAI: An AI-Based Framework for Detecting Illegal Mining and Deforestation Using Sentinel-2 Satellite Imagery"

Submitted by

MS. BARKARE VEDIKA NANDKISHOR **EXAM NO. : B401140084**

MS. PATIL PRANJALI MOHAN **EXAM NO. : B401140112**

MR. PATIL SHUBHAM DNYANESHWAR **EXAM NO. : B401140114**

MR. SONAWANE PRAVIN RAJESH **EXAM NO. : B401140124**

is a bonafide project work carried out under the supervision of **Ms. S. P. Baviskar** and is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** in Artificial Intelligence (AI) and Data Science.

(Ms. S. P. Baviskar)
Guide, Dept. of Artificial Intelligence (AI) and Data Science

(Ms. D. D. Survase)
Head, Dept. of Artificial Intelligence (AI) and Data Science

(Dr. M. V. Bhatkar)
Principal

Signature of External Examiner: I

Signature of External Examiner: II

Place : Nashik

Date :

ISO 21001:2018 Certified | NAAC Accredited

Jawahar Education Society's,
INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NASHIK.
(Approved by AICTE, New Delhi, DTE, Government of Maharashtra, Affiliated to Savitribai Phule Pune University)

PROJECT APPROVAL SHEET

A Project Report Titled as

"GREENVISIONAI: An AI-Based Framework for Detecting Illegal Mining and Deforestation Using Sentinel-2 Satellite Imagery"

Is verified for its originality in documentation, problem statement, proposed work and implementation successfully completed by

MS. BARKARE VEDIKA NANDKISHOR	EXAM NO. : B401140084
MS. PATIL PRANJALI MOHAN	EXAM NO. : B401140112
MR. PATIL SHUBHAM DNYANESHWAR	EXAM NO. : B401140114
MR. SONAWANE PRAVIN RAJESH	EXAM NO. : B401140124

at

DEPARTMENT OF ARTIFICIAL INTELLIGENCE (AI) AND DATA SCIENCE

JESITMR, Nashik

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

ACADEMIC YEAR 2025-2026

(Ms. S. P. Baviskar)	**(Ms. D. D. Survase)**
Guide, Dept. of Artificial Intelligence (AI) and Data Science	Head, Dept. of Artificial Intelligence (AI) and Data Science

ACKNOWLEDGEMENT

It is my immense pleasure to work on this project titled *"GREENVISIONAI: An AI-Based Framework for Detecting Illegal Mining and Deforestation Using Sentinel-2 Satellite Imagery."* This project has provided me with an opportunity to develop a deeper understanding of the subject.

I would like to express my sincere gratitude to *Dr. M. V. Bhatkar*, Principal of Jawahar Education Society's Institute of Technology, Management and Research, Nashik, for providing me the opportunity to carry out this project.

I am also thankful to *Ms. D. D. Survase*, Head of the Department of Artificial Intelligence (AI) and Data Science, for his valuable encouragement and support throughout the various phases of my project work.

I would like to extend my heartfelt thanks to my project guide, *Ms. S. P. Baviskar*, for encouraging me to explore this subject and for providing valuable guidance and continuous support during the preparation of this project.

Additionally, I express my sincere thanks to all the faculty members of the Department of Artificial Intelligence (AI) and Data Science for their cooperation and support, which contributed to the successful completion of this project.

MS. BARKARE VEDIKA NANDKISHOR

MS. PATIL PRANJALI MOHAN

MR. PATIL SHUBHAM DNYANESHWAR

MR. SONAWANE PRAVIN RAJESH

ABSTRACT

Illegal mining and deforestation are serious environmental threats that cause biodiversity loss, ecological imbalance, soil degradation, habitat destruction and accelerated climate change. Traditional monitoring methods depend on field surveys, manual inspection and government patrols. These approaches are costly, time consuming and difficult to scale across large forest regions. Recent advances in satellite remote sensing and artificial intelligence provide an effective alternative for continuous observation of forest ecosystems and mining affected areas.

The proposed system, GreenVisionAI, is an AI-based environmental monitoring framework for detecting deforestation and illegal mining activity using Sentinel-2 satellite imagery. The system uses Google Earth Engine for satellite data acquisition and preprocessing, Random Forest classification for forest cover analysis and a U-Net based convolutional neural network for mining region segmentation. The framework compares multi-temporal satellite imagery to estimate forest cover change and applies deep learning based segmentation to highlight mining footprints, heatmaps and severity indicators.

GreenVisionAI is implemented as an interactive web application. Users can enter latitude and longitude coordinates or select predefined study regions. The backend, developed using Flask and Python, communicates with Google Earth Engine and machine learning modules. The frontend, developed using React.js, provides a map-based interface, output panels, visual overlays and result summaries. The system supports scalable, cost effective and accessible monitoring of ecological degradation by combining AI, cloud geospatial processing and satellite imagery.

Keywords: *GreenVisionAI, Satellite Imagery, Sentinel-2, Deforestation Detection, Illegal Mining Detection, Random Forest, U-Net CNN, Google Earth Engine, Remote Sensing, Environmental Monitoring.*

Table of Contents

List of Abbreviations		i
List of Figures		ii
List of Tables		iii
01	**Introduction**	1
	1.1 Background	1
	1.2 Motivation	1
	1.3 Problem Definition	2
	1.4 Organization of Report	3
02	**Literature Survey**	4
03	**Requirement Analysis Phase**	6
	3.1 Introduction	6
	3.1.1 Project Scope	6
	3.1.2 User Classes and Characteristics	7
	3.1.3 Assumptions and Dependencies	8
	3.2 Functional Requirements	8
	3.3 External Interface Requirements	10
	3.4 Non-Functional Requirements	11
	3.5 System Requirements	12
	3.6 Analysis Models: SDLC Model	16
	3.7 System Implementation Plan	13
	3.8 System Economic Feasibility	14
04	**Design Phase**	15
	4.1 System Architecture	15
	4.2 Data Flow Diagrams	18
	4.3 Entity-Relationship Diagram	21
	4.4 UML Diagrams	22
05	**Technical Specifications**	32
	5.1.1 Technology Details Used in the Project	32
06	**Planning Phase**	34
	6.1 Project Estimation	34
	6.2 Project Schedule and Team Organization	36
07	**Software Implementation**	37
	7.1 Introduction	37
	7.2 Project Structure	38

	7.3 Satellite Data Processing Pipeline	38
	7.4 Environment Configuration	39
	7.5 Business Logic	40
08	**Software Testing**	43
	8.1 Introduction	43
	8.1.2 Test Case for Location Input	43
	8.1.3 Test Case for Deforestation Detection	44
	8.1.4 Test Case for Mining Detection	44
	8.2 Snapshot of Test Case	45
09	**Result**	46
	9.1 Snapshots of the Results	46
	9.2 Observation and Evaluation	48
10	**Conclusions and Future Work**	49
	10.1 Conclusions	49
	10.2 Future Work	50
	References	52
	Glossary	54
	ANNEXURE A - Literature Review Table	56
	ANNEXURE B - Code Snippets	59
	ANNEXURE C - User Manual	63
	ANNEXURE D - Plagiarism Report	66
	ANNEXURE E - Publication and Certificates	76

LIST OF ABBREVIATIONS

ABBREVIATION	ILLUSTRATION
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
DL	Deep Learning
GEE	Google Earth Engine
GIS	Geographic Information System
HTTP	HyperText Transfer Protocol
IoU	Intersection over Union
JSON	JavaScript Object Notation
ML	Machine Learning
NDVI	Normalized Difference Vegetation Index
NIR	Near Infrared
RGB	Red Green Blue
REST	Representational State Transfer
RF	Random Forest
ROI	Region of Interest
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
SWIR	Short Wave Infrared
UML	Unified Modeling Language
UI	User Interface
UX	User Experience

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
1:	System Architecture	17
2:	DFD Level 0 – Context Diagram	19
3:	DFD Level 1 – Detailed Data Flow	20
4:	ER Diagram	22
5:	Use Case Diagram	23
6:	Class Diagram	24
7:	Activity Diagram	25
8:	Sequence Diagram	26
19:	Object Diagram	27
10:	Collaboration Diagram	28
11:	State Chart Diagram	29
12:	Component Diagram	30
13:	Deployment Diagram	31
14:	Deforestation Detection Output	46
15:	Forest Cover Change Panel	47
16:	Illegal Mining Detection Output	47
17:	Mining Heatmap and Severity Score	48

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
1	Project Development Mode	35
2	Development Activities	36
3	GreenVisionAI Assistant Team	36
4	API Endpoints	42
5	Test Cases for Location Input	43
6	Test Cases for Deforestation Detection	44
7	Test Cases for Mining Detection	45
8	Comparative Performance of Detection Methods	48

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND

The rapid growth of industrialization, urban expansion, and resource extraction activities has significantly increased pressure on natural ecosystems across the world, and among the most critical environmental challenges are deforestation and illegal mining, both of which contribute to biodiversity loss, land degradation, water pollution, and accelerated climate change. Traditional environmental monitoring methods primarily rely on field surveys, manual inspections, and government patrols, which are often expensive, time-consuming, and difficult to implement over large or remote geographical regions, and as a result environmental damage is frequently detected only after substantial ecological degradation has already occurred. Breakthroughs in satellite remote sensing have transformed this picture: missions such as Sentinel-2 provide high-resolution multispectral imagery with regular revisit cycles, while cloud-based geospatial platforms such as Google Earth Engine remove the traditional burden of downloading and locally processing large satellite archives. This project applies these advances to build GreenVisionAI, an intelligent environmental monitoring system capable of detecting both deforestation and illegal mining activity from Sentinel-2 imagery, and it implements two coordinated analysis modules – a Random Forest based forest-cover classification pipeline and a U-Net based mining-segmentation pipeline – where satellite bands are processed into model-ready composites so the system returns location-specific, evidence-grounded results through an interactive FastAPI and Streamlit based web platform.[2][3]

1.2 MOTIVATION

GreenVisionAI was developed in response to a set of recurring practical challenges observed across forest-monitoring and mining-surveillance workflows, and these repeated issues shaped the system's functional priorities, implementation choices, and ongoing refinement process.[1][2]

- **Field-Survey Dependence:** Ground patrols and manual inspection provide reliable information but are slow, costly, and limited by accessibility. A system that can monitor large regions automatically would dramatically reduce response time.
- **Limitations of Manual Interpretation:** Visual interpretation of raw satellite

imagery fails to scale across large forest and mining belts. Analysts need automated classification that does not depend on exact visual cues or prior site knowledge.[4]

- **Single-Task System Constraints:** Standalone deforestation-tracking tools cannot detect mining activity, and standalone mining-detection tools cannot track forest cover change. Combining both tasks within one architecture addresses this gap directly.[2][3]
- **Data Processing Needs:** Beyond raw imagery, users frequently need processed outputs such as forest-cover percentage, change-detection maps, and severity scores. Integrating both analysis types into a single pipeline provides a unified solution.[5][7]
- **Tool Fragmentation:** Organizations often track environmental data across multiple disconnected tools and spreadsheets. A system that handles location input, image retrieval, and dual-task analysis through one interface eliminates the need for multiple separate tools.
- **Accessibility:** A web-based solution accessible from any browser democratizes access to AI-powered environmental monitoring without requiring specialized GIS software installation.

1.3 PROBLEM DEFINITION

Detecting deforestation and illegal mining across large and often remote geographical regions presents a range of difficulties for environmental analysts and administrative authorities, because relying on manual monitoring methods introduces a broad and interconnected set of obstacles in routine environmental surveillance.

- Manual monitoring of forests and mining regions is slow and difficult to scale.
- Existing systems often focus on either deforestation or mining detection, but not both.
- Satellite data requires significant preprocessing before it can be used for practical analysis.
- Environmental analysts need visual and quantitative outputs such as forest cover percentage, change-detection maps, and mining severity indicators.
- Non-technical users require a simple interface to enter a location and obtain meaningful results without GIS expertise.

- Lack of transparency in how a detection result was derived makes it difficult for authorities to act confidently on AI-generated outputs.

To address these challenges, the proposed system delivers the following capabilities:

- Automated Sentinel-2 image acquisition and preprocessing through Google Earth Engine.
- Random Forest based forest-cover classification with multi-temporal change detection.
- U-Net based mining-region segmentation with overlay generation and severity scoring.
- A combined, dual-task analysis pipeline within a single application.
- Flexible location input – latitude/longitude entry or predefined example regions – through a single web interface.
- A user-friendly Streamlit interface, backed by a FastAPI service layer, requiring no GIS expertise.

1.4 ORGANIZATION OF REPORT

This report is organized into the following chapters:
Chapter 1 – Introduction: project background, motivation, and problem definition.
Chapter 2 – Literature Survey: review of remote sensing, Random Forest classification, U-Net segmentation, and Google Earth Engine.
Chapter 3 – Requirement Analysis Phase: functional and non-functional requirements, scope, and system requirements.
Chapter 4 – Design Phase: system architecture, data flow diagrams, and UML models.
Chapter 5 – Technical Specifications: technology stack and implementation details.
Chapter 6 – Planning Phase: COCOMO estimation, project schedule, and team structure.
Chapter 7 – Software Implementation: project structure, satellite data processing pipeline, and business logic.
Chapter 8 – Software Testing: test cases and validation results.
Chapter 9 – Result: system snapshots and performance observations.
Chapter 10 – Conclusions and Future Work: project summary and directions for further development.

CHAPTER 2: LITERATURE SURVEY

Research into AI-based environmental monitoring has gained momentum as remote sensing, cloud computing, and machine learning progress, and this chapter surveys the foundational concepts and enabling technologies that underpin the proposed system.[1][2]

SATELLITE REMOTE SENSING AND LAND COVER MONITORING: Remote sensing has been widely used for forest monitoring and land cover analysis, allowing researchers to observe large geographic regions repeatedly and identify temporal changes without physical access to the site. Satellite imagery is captured at regular intervals, enabling continuous tracking of vegetation health, land degradation, and surface disturbance across regions that would otherwise require extensive field travel to survey.[4][5]

VEGETATION INDICES AND SPECTRAL ANALYSIS: Spectral vegetation indices such as the Normalized Difference Vegetation Index (NDVI) compare reflectance in the red and near-infrared bands to distinguish healthy vegetation from degraded or bare land. Such indices provide a quantitative foundation for automated deforestation analysis and remain a standard reference point against which newer machine-learning-based classification methods are evaluated.[3]

RANDOM FOREST FOR LAND COVER CLASSIFICATION: Machine learning methods have shown strong performance in land cover classification, and Random Forest is widely used because it can handle high-dimensional spectral data, requires limited parameter tuning, and provides robust results even with moderately sized training sets. For forest cover mapping, Random Forest can classify pixels into categories such as dense forest, degraded forest, agricultural land, bare soil, and water body, and is well suited to comparing historical and recent imagery for change detection.[6][7]

U-NET FOR PIXEL-WISE SEGMENTATION: Deep learning methods have improved image segmentation tasks considerably. U-Net is a convolutional neural network architecture originally proposed for biomedical image segmentation, but it has since been successfully adapted to remote sensing applications. Its encoder-decoder structure and skip connections help preserve fine spatial detail while learning high-level contextual features, allowing it to generate pixel-wise masks that highlight probable mining footprints

with greater precision than threshold-based methods.[9]

GOOGLE EARTH ENGINE FOR CLOUD GEOSPATIAL PROCESSING: Google Earth Engine provides the orchestration layer for satellite-based analysis, offering composable building blocks – image collections, filtering operations, band-math expressions, and export utilities – for constructing reproducible geospatial processing pipelines. It provides access to public satellite archives and allows filtering, compositing, and processing of images over large areas without requiring local download of raw data.[10]

MULTI-TEMPORAL CHANGE DETECTION: Researchers are actively using paired or time-series satellite composites to detect land-cover change over time. Common approaches include image differencing, post-classification comparison, and ratio-based change indices, while for forest monitoring specifically, comparing classified forest percentage between two dates provides a direct, interpretable measure of cover loss or gain.[7]

WEB-BASED GEOSPATIAL APPLICATIONS: Modern Python web frameworks make it practical to build responsive, AI-enhanced geospatial applications accessible through any browser. FastAPI enables efficient, asynchronous server-side request handling for image-processing and model-inference endpoints, while Streamlit allows rapid construction of interactive data-visualisation interfaces directly from Python scripts, together enabling real-time location input, map display, and result rendering without separate frontend tooling.[8]

GAPS AND OPPORTUNITIES: Most existing tools handle either deforestation tracking or mining detection in isolation, and many rely solely on threshold-based spectral indices rather than learned classifiers, which limits their accuracy in visually complex terrain. Unified platforms that combine classical machine learning with deep segmentation within a single interface remain uncommon, and systems that omit clear severity scoring make prioritising follow-up action difficult, so the proposed system targets these gaps by offering:

- Combined deforestation and mining detection within a single application.
- A learned Random Forest classifier rather than a fixed spectral threshold for forest classification.
- A U-Net segmentation model for pixel-wise mining footprint detection rather than coarse bounding boxes.

CHAPTER 3: REQUIREMENT ANALYSIS PHASE

3.1 INTRODUCTION

This chapter formally documents the functional and non-functional requirements that govern the proposed system. It defines what the system must do, captures expected behaviors, operational constraints, interface specifications, and performance targets that guide the design and development process. The system under specification is a web-based, AI-powered environmental monitoring platform allowing users to enter geographic coordinates and obtain deforestation and illegal mining analysis derived from Sentinel-2 satellite imagery. This document serves as the authoritative reference for all subsequent design, implementation, and verification activities.[16][17]

3.1.1 PROJECT SCOPE

The system is a comprehensive AI tool that enables location-based environmental analysis across two detection tasks and bridges the gap between raw satellite imagery and actionable insight by extracting, classifying, and presenting results tailored to analyst and authority needs.[1][10][12]

SCOPE OF THE PROJECT INCLUDES:

- Development of a web-based application accessible from desktop and mobile browsers.
- Interactive coordinate-based and predefined-region location selection.
- Sentinel-2 image retrieval through Google Earth Engine with date-range and cloud-cover filtering.
- Multispectral band selection and cloud-free composite generation.
- Random Forest based forest-cover classification.
- Multi-temporal change detection comparing historical and current forest cover.
- U-Net based mining-region segmentation.
- Mining overlay, heatmap, and severity-score generation.

- Form-based query processing through a FastAPI backend.
- Interactive result visualisation through a Streamlit interface.
- Responsive user interface with real-time processing feedback.

OUT OF SCOPE:

- Real-time satellite alerting for sudden land-cover change (future enhancement).
- Multi-user authentication and historical query management.
- Persistent storage of analysis results across sessions.

3.1.2 USER CLASSES AND CHARACTERISTICS

This system serves the following user categories:

1. ENVIRONMENTAL ANALYST

- **Description:** Professional users seeking to monitor land-cover and mining-affected regions.
- **Responsibilities:** Selecting locations, running detection analysis, interpreting results.
- **Technical Skill Level:** Basic to intermediate GIS literacy.
- **Needs:** Quick, accurate detection results with clear visual overlays.

2. GOVERNMENT OFFICER / ADMINISTRATOR

- **Description:** Users relying on system outputs as decision support for inspection planning.
- **Responsibilities:** Reviewing severity scores, prioritising follow-up field verification.
- **Technical Skill Level:** Intermediate; familiar with administrative reporting.
- **Needs:** Severity-ranked outputs and exportable summary statistics.

3. DEVELOPER / MAINTAINER

- **Description:** Technical personnel maintaining the system.
- **Responsibilities:** System maintenance, model updates, API integration.
- **Technical Skill Level:** Advanced; proficient in Python, FastAPI, and Streamlit.[16][17]

3.1.3 ASSUMPTIONS AND DEPENDENCIES

ASSUMPTIONS:

- Users have access to internet connectivity and a modern web browser.
- Sentinel-2 imagery is available for the selected region and time period.
- Cloud-cover filtering yields a usable, sufficiently clear composite image.
- Google Earth Engine services remain available and accessible.

DEPENDENCIES:

- Google Earth Engine API for satellite data acquisition.
- Trained Random Forest model for forest-cover classification.
- Trained U-Net model for mining-region segmentation.
- scikit-learn library for Random Forest implementation.
- PyTorch library for U-Net implementation and inference.
- FastAPI framework for backend API routing.
- Streamlit framework for the frontend interface.
- NumPy and OpenCV for image-array operations and visualisation support.[18][19]

3.2 FUNCTIONAL REQUIREMENTS

3.2.1 LOCATION INPUT AND SATELLITE DATA ACQUISITION

- The system shall allow users to enter latitude and longitude coordinates or choose predefined example regions.
- The system shall acquire Sentinel-2 satellite imagery using Google Earth Engine.
- The system shall filter images by date range, region of interest, and cloud-cover percentage.
- The system shall generate cloud-free median composites from filtered image collections.

- The system shall select relevant spectral bands (B2, B3, B4, B8, B11, B12) for analysis.
- The system shall pass processed composites to the appropriate detection module.
- The system shall validate coordinate input ranges before triggering analysis.
- The system shall display processing status while satellite data is being retrieved.[20][21]

3.2.2 DEFORESTATION DETECTION

- The system shall classify forest and non-forest areas using a Random Forest model.
- The system shall compare historical and recent imagery to estimate forest-cover change.
- The system shall calculate forest-cover percentage for each selected time period.
- The system shall display the percentage change and relative forest loss or gain.[12]

3.2.3 ILLEGAL MINING DETECTION

- The system shall accept the processed satellite composite and apply a U-Net segmentation model.
- The system shall generate a pixel-wise probability mask for mining-affected regions.
- The system shall threshold the probability mask to produce a binary mining overlay.
- The system shall calculate a severity score based on affected area and model confidence.
- The system shall generate a heatmap visualisation of mining probability.
- The system shall support analysis of multiple candidate regions within one session.[22]

3.2.4 RESULT VISUALIZATION

- The system shall display satellite image comparisons between two time periods.
- The system shall render classified forest-cover outputs and mining overlays on the interface.
- The system shall display probability heatmaps for mining detection.

- The system shall present summary statistics including forest-cover percentage and severity score.
- The system shall provide expandable detail panels for each analysis result.[13][23]

3.3 EXTERNAL INTERFACE REQUIREMENTS

3.3.1 USER INTERFACE

We implemented a web-based interface using Streamlit with the following components:

- Coordinate input form with example-region quick-select.
- Real-time processing status indicators.
- Side-by-side satellite image comparison panels.
- Mining overlay and heatmap display panels.
- Interactive summary statistics and severity-score display.[16]

3.3.2 SOFTWARE INTERFACE

The system communicates through RESTful APIs:

- Frontend (Streamlit) sends HTTP requests to backend (FastAPI) endpoints.
- Backend processes requests and communicates with Google Earth Engine and ML modules.
- Responses are returned in JSON format.
- Coordinate and mode parameters are validated using Pydantic models.[15]

3.3.3 HARDWARE INTERFACE

The system runs on standard devices capable of executing Python backend services, model inference, and the Streamlit web application. GPU support improves U-Net inference speed but is not mandatory for basic demonstration.[17]

3.3.4 COMMUNICATION INTERFACE

- HTTPS for secure client-server communication.

- CORS configuration for cross-origin requests between Streamlit and FastAPI.
- Internet access for Google Earth Engine and map-tile services.[24]

3.4 NON-FUNCTIONAL REQUIREMENTS

3.4.1 PERFORMANCE REQUIREMENTS

- Satellite data retrieval should complete within a reasonable time for a moderate study area.
- Forest classification responses should be generated within a few minutes depending on data availability.
- The system should handle Sentinel-2 composites for typical region-of-interest sizes.
- The interface should remain responsive during processing.[16]

3.4.2 SECURITY REQUIREMENTS

- Google Earth Engine service-account credentials must be stored securely in environment variables.
- API keys and backend configuration must not be exposed to the frontend.
- Input sanitisation to prevent invalid coordinate or parameter injection.
- HTTPS encryption for data in transit.[24]

3.4.3 USABILITY REQUIREMENTS

- Intuitive interface requiring no GIS training.
- Clear error messages for invalid coordinates or unavailable imagery.
- Visual feedback for all user actions.
- Responsive design for various screen sizes.[17]

3.4.4 RELIABILITY REQUIREMENTS

- Graceful error handling for Earth Engine API failures.
- Fallback messaging when imagery is unavailable for a selected region.
- Automatic retry for transient network errors.[17]

3.5 SYSTEM REQUIREMENTS

3.5.1 SOFTWARE REQUIREMENTS

- **Frontend:** Streamlit, Python.
- **Backend:** FastAPI, Python, Uvicorn.
- **AI/ML:** scikit-learn, PyTorch, Google Earth Engine API.
- **Browser:** Chrome, Firefox, Edge (latest versions).[25]

3.5.2 HARDWARE REQUIREMENTS

Client Side:

- Any device with a modern web browser.
- Minimum 2GB RAM, stable internet connection.

Server Side:

- Minimum 4-core CPU, 8GB RAM (16GB recommended).
- 10GB storage for application, models, and temporary files.
- Network connectivity for Earth Engine API access; GPU optional for faster inference.[17]

3.6 ANALYSIS MODELS: SDLC MODEL

The project adopts **Agile Development** as its SDLC model. Structuring work into focused iterative sprints, the model provides an organising framework for development tasks and a predictable rhythm for team work.[26]

- Rapid prototyping and feedback incorporation.
- Incremental feature development.
- Continuous testing and integration.
- Flexibility to adapt to changing requirements.[27]

Development phases include:

1. **Sprint 1:** Basic coordinate input and Sentinel-2 image retrieval.
2. **Sprint 2:** Random Forest model integration for forest classification.
3. **Sprint 3:** Forest-cover change detection and percentage calculation.
4. **Sprint 4:** U-Net model integration for mining segmentation.
5. **Sprint 5:** Heatmap and severity-score generation.
6. **Sprint 6:** Streamlit UI polish and testing.

3.7 SYSTEM IMPLEMENTATION PLAN

Phase 1: Environment Setup

- Configure FastAPI backend with Uvicorn.
- Set up Streamlit frontend application.
- Configure Google Earth Engine service-account credentials.

Phase 2: Core Satellite Pipeline

- Implement Sentinel-2 image retrieval service.
- Set up cloud-filtering and composite-generation logic.
- Create band-selection and preprocessing utilities.

Phase 3: Deforestation Module

- Train and integrate the Random Forest classifier.
- Implement forest-cover percentage calculation.
- Build multi-temporal change-detection logic.

Phase 4: Mining Detection Module

- Train and integrate the U-Net segmentation model.
- Implement mask thresholding and overlay generation.
- Add severity-score calculation logic.

Phase 5: Frontend Integration

- Build coordinate-input and example-region selection components.
- Implement side-by-side satellite comparison display.
- Add mining overlay and heatmap rendering.
- Display summary statistics and severity scores.
- Connect the Streamlit frontend to FastAPI endpoints.

Phase 6: Testing and Deployment

- Perform integration testing across selected case-study regions.
- Optimise processing performance.
- Deploy application.

3.8 SYSTEM ECONOMIC FEASIBILITY

GreenVisionAI uses open-source technologies for the majority of its components, significantly reducing development cost. The primary external dependency is Google Earth Engine, which is freely available for academic and research use and remains affordable even for moderate satellite-processing workloads. FastAPI, Streamlit, scikit-learn, and PyTorch are freely available under open-source licenses. Hosting requires standard web-server infrastructure without specialised GPU hardware for basic demonstration, keeping operational costs manageable for both prototype and production deployments.[19][20]

CHAPTER 4: DESIGN PHASE

4.1 SYSTEM ARCHITECTURE

We implemented a three-tier architecture integrated with a dual-pipeline satellite analysis workflow to enforce a clear separation of concerns, enabling independent development, scaling, and maintenance of each tier.[1][17]

4.1.1 GREENVISIONAI ARCHITECTURE OVERVIEW

Processing within GreenVisionAI proceeds through four distinct stages:

1. Location and Data Ingestion:

- Users submit latitude and longitude coordinates, or select a predefined example region, through the Streamlit interface.
- The backend constructs a region-of-interest geometry from the submitted coordinates.
- Sentinel-2 Level-2A imagery is queried from Google Earth Engine for the specified region and date range.
- Cloud-cover filtering removes unsuitable scenes before compositing.

2. Preprocessing and Composite Generation:

- A cloud-free median composite is generated from the filtered image collection.
- Relevant spectral bands (B2, B3, B4, B8, B11, B12) are selected from the composite.
- The processed composite is converted into a model-ready array for inference.

3. Dual-Task Model Inference:

- The Random Forest classifier processes the composite for forest-cover classification.
- The U-Net model processes the composite for mining-region segmentation.
- Both models execute independently within the same request when both analyses are requested.

4. Result Generation:

- Forest-cover percentage and change values are computed from classification output.
- Mining overlays, heatmaps, and severity scores are computed from segmentation output.
- Final results are returned to the Streamlit frontend for visualisation.

4.1.2 THREE-TIER ARCHITECTURE

Presentation Layer (Frontend):

- Built with Streamlit and Python.
- Provides coordinate input, example-region selection, and result panels.
- Components: LocationInputPanel, ComparisonView, MiningOverlayPanel, SummaryStatsPanel.
- Communicates with backend via REST APIs.

Application Layer (Backend):

- FastAPI framework with Uvicorn ASGI server.
- Services: GEEService, DeforestationService, MiningService.
- Handles request validation, satellite-data orchestration, and model invocation.
- Integrates with Google Earth Engine and the trained ML/DL models.

Data Layer:

- Serialised Random Forest model (random_forest.pkl) loaded at startup.
- Serialised U-Net model weights (unet_model.pth) loaded at startup.
- Temporary file storage for generated overlays and output images.[18][21]

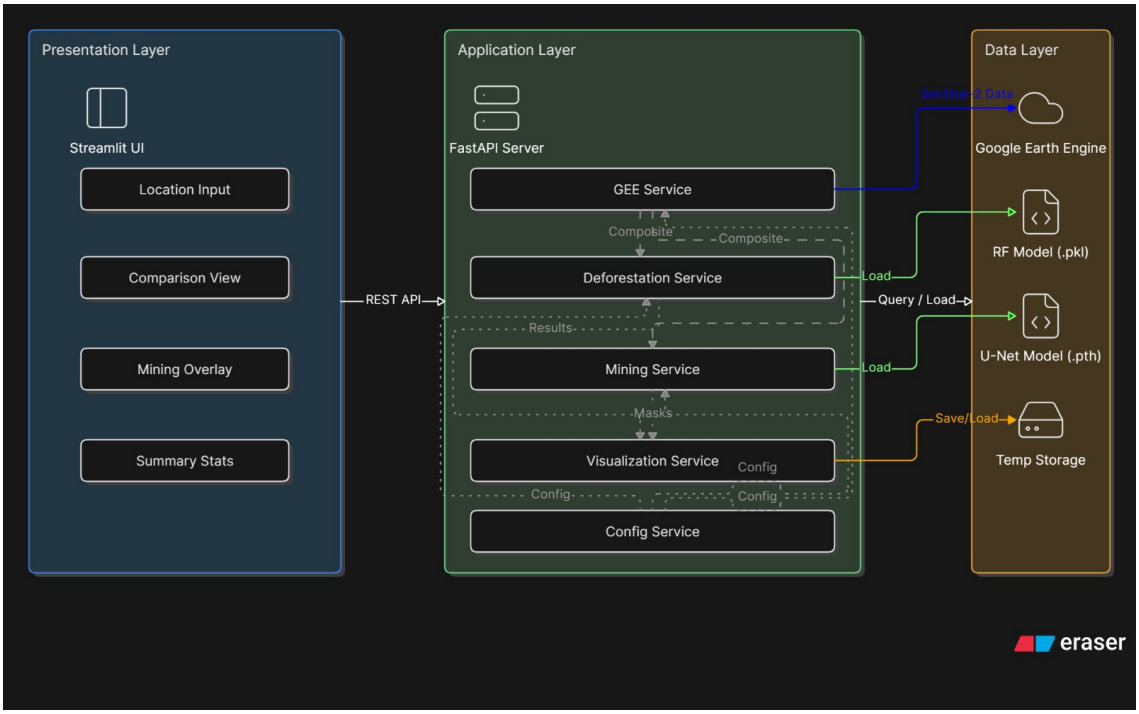


Figure 1: System Architecture

4.1.3 KEY COMPONENTS

GEE Service (gee_service.py):

- Authenticates with Google Earth Engine using service-account credentials.
- Constructs the region-of-interest geometry from submitted coordinates.
- Filters the Sentinel-2 collection by date range and cloud-cover percentage.
- Generates the cloud-free median composite and selects required bands.
- Returns the processed composite as a model-ready array.

Deforestation Service (deforestation_service.py):

- Loads the trained Random Forest model and runs classification on the composite.
- Calculates forest-cover percentage and multi-temporal change.

Mining Service (mining_service.py):

- Loads the trained U-Net model and runs segmentation inference.
- Thresholds the probability mask and computes the severity score.

Visualization Service:

- Generates colour-mapped overlay images from segmentation masks.
- Generates probability heatmaps for mining detection results.
- Renders side-by-side satellite comparison images for deforestation results.
- Formats summary statistics for display in the Streamlit interface.

Config Service:

- Loads Google Earth Engine service-account credentials from environment variables.
- Resolves file paths for the trained Random Forest and U-Net model files.
- Manages default date ranges and cloud-cover thresholds.
- Provides centralized configuration access to all backend services.[20][22]

4.2 DATA FLOW DIAGRAMS

4.2.1 DFD LEVEL 0 (CONTEXT DIAGRAM)

At the highest level of abstraction, the context diagram presents the entire system as a single processing unit and maps all interactions between that unit and external entities.[16]

External Entities:

- **User:** Submits location coordinates, selects analysis mode, receives results.
- **Google Earth Engine:** Provides Sentinel-2 satellite imagery and geospatial processing.

Data Flows:

- User $\rightarrow$ System: Coordinates, date range, analysis mode selection.
- System $\rightarrow$ User: Classified outputs, overlays, heatmaps, severity scores.
- System $\leftrightarrow$ Google Earth Engine: Region query, filtered imagery, composite generation.

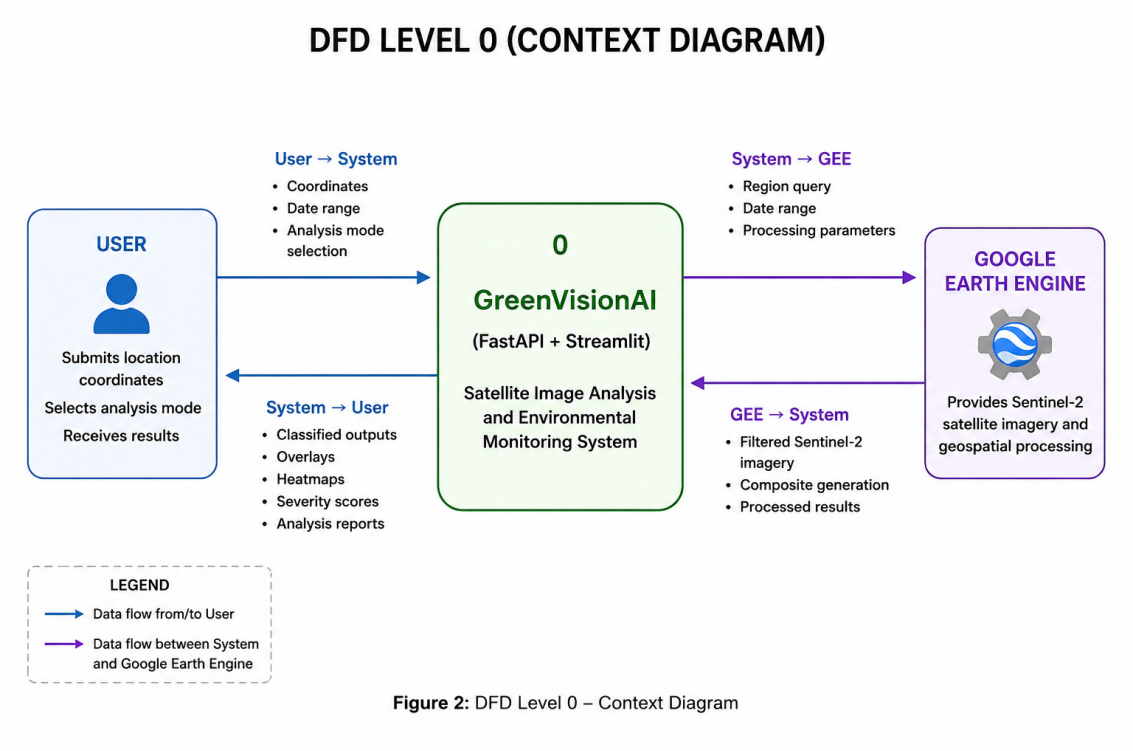


Figure 2: DFD Level 0 – Context Diagram

4.2.2 DFD LEVEL 1 (SYSTEM DECOMPOSITION)

The Level 1 DFD expands the context diagram by decomposing the system into its principal processing elements.

Process 1: Satellite Data Acquisition

- Input: User coordinates, date range, cloud-cover threshold.
- Processing: Region creation, image filtering, composite generation, band selection.
- Output: Model-ready Sentinel-2 composite.

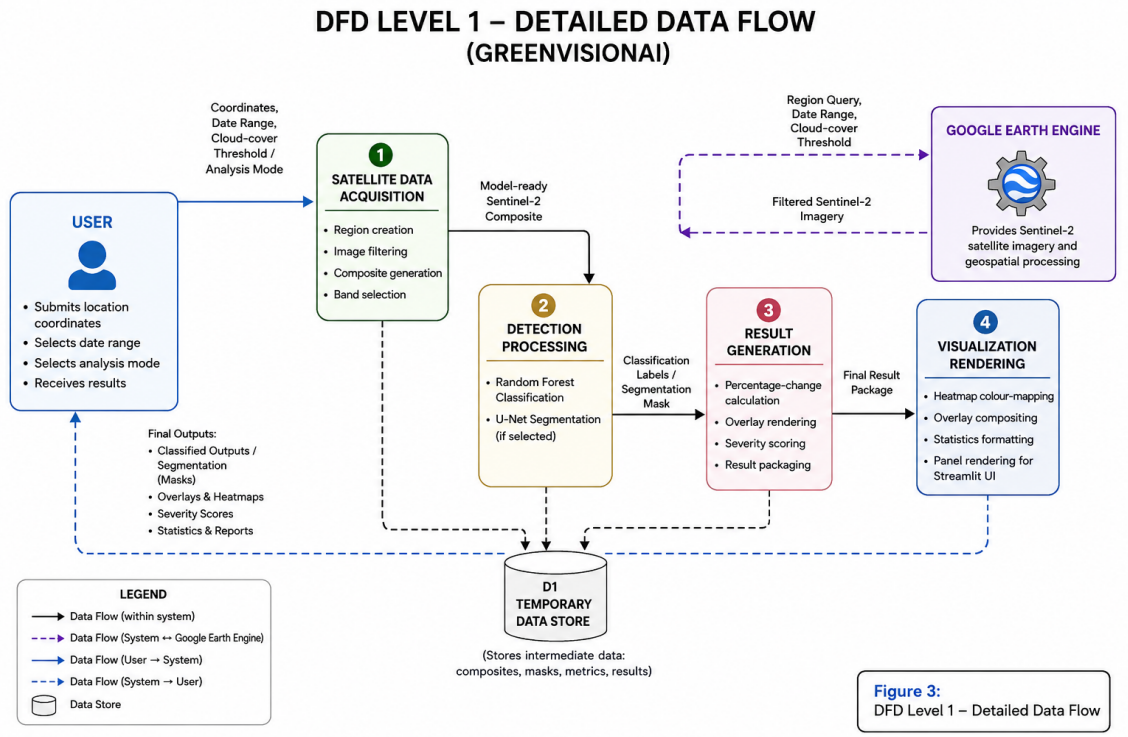


Figure 3: DFD Level 1 – Detailed Data Flow

Process 2: Detection Processing

- Input: Processed satellite composite.
- Processing: Random Forest classification and/or U-Net segmentation.
- Output: Classification labels or segmentation mask.

Process 3: Result Generation

- Input: Classification labels or segmentation mask.
- Processing: Percentage-change calculation, overlay rendering, severity scoring.
- Output: Final result package returned to the frontend.

Process 4: Visualization Rendering

- Input: Final result package and original satellite composite.
- Processing: Heatmap colour-mapping, overlay compositing, statistics formatting.
- Output: Rendered panels displayed in the Streamlit interface.

4.3 ENTITY-RELATIONSHIP DIAGRAM

The system does not require persistent storage for the demonstration version. We express the underlying data model in terms of conceptual entities and their associations for future extension.[16]

Key Entities:

- **Location:** latitude, longitude, regionName, requestedAt.
- **SatelliteImage:** sourceCollection, dateRange, cloudCoverPercent, bands.
- **Composite:** compositeId, locationRef, generatedAt.
- **ClassificationResult:** forestPercentOld, forestPercentNew, changePercent.
- **SegmentationResult:** maskPath, severityScore, affectedAreaPercent.
- **AnalysisRequest:** requestId, mode, status, timestamp.
- **Result:** resultId, requestRef, overlayPath, summaryStats.

Relationships:

- Location has many AnalysisRequests (1:N).
- AnalysisRequest has one Composite (1:1).
- Composite has one ClassificationResult or SegmentationResult (1:1).
- AnalysisRequest produces one Result (1:1).
- Result may reference one SatelliteImage source (1:1).

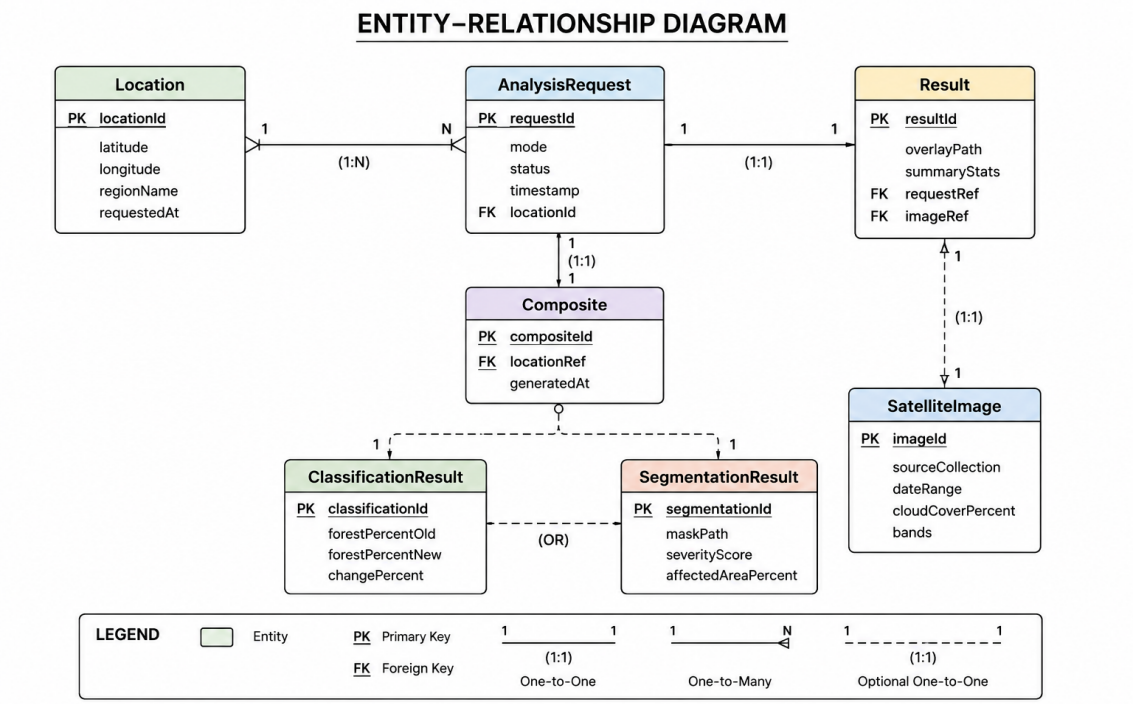


Figure 4: Entity Relationship Diagram

Figure 4: Entity Relationship Diagram

4.4 UML DIAGRAMS

4.4.1 USE CASE DIAGRAM

The use case diagram captures the complete set of actions available to the user and their direct interactions with system functionality.[16]

Actor: User

Use Cases:

- Enter Location Coordinates
- Select Predefined Example Region
- Run Deforestation Detection
- Run Illegal Mining Detection
- View Forest Cover Comparison
- View Mining Overlay and Heatmap
- View Severity Score

- View Summary Statistics
- Reset Analysis Session

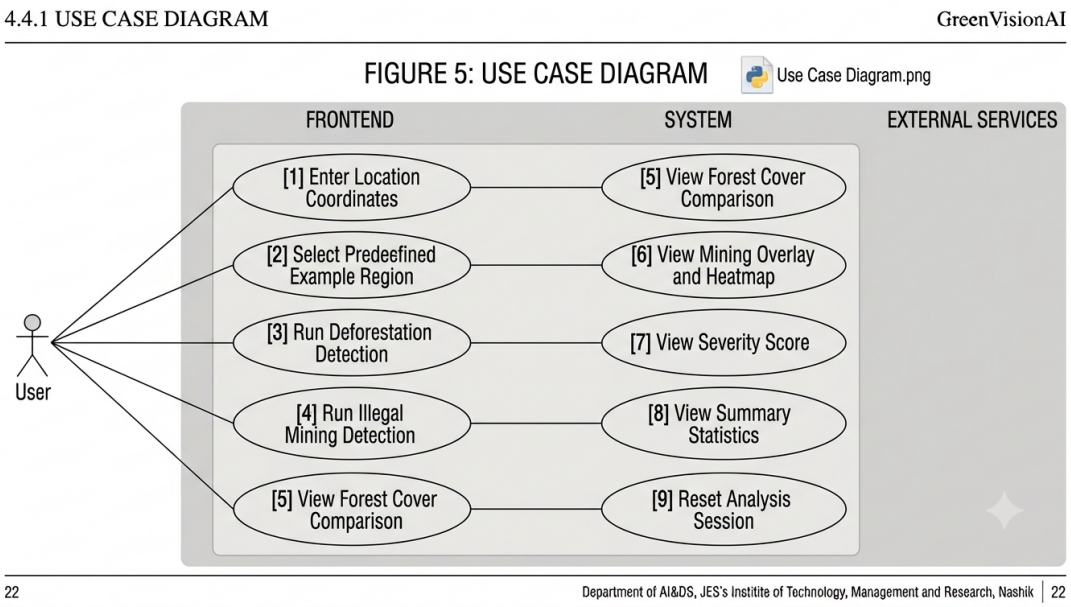


Figure 5: Use Case Diagram

4.4.2 CLASS DIAGRAM

The class diagram presents the principal software classes that constitute the system, grouped by tier.

Backend Classes:

- **GEEService:** `region`, `dateRange`, `fetchComposite()`, `filterCloudCover()`, `selectBands()`
- **DeforestationService:** `model`, `classify()`, `calculateChange()`
- **MiningService:** `model`, `segment()`, `calculateSeverity()`
- **VisualizationService:** `generateOverlay()`, `generateHeatmap()`
- **AnalysisRoutes:** `analyzeRegion()`, `getStatus()`, `reset()`
- **ConfigService:** `loadCredentials()`, `getModelPaths()`

Frontend Classes:

- **LocationInputPanel:** `latitude`, `longitude`, `mode`, `submitRequest()`

- **ComparisonView:** `oldImage`, `newImage`, `renderComparison()`
- **MiningOverlayPanel:** `overlayImage`, `heatmap`, `renderOverlay()`
- **SummaryStatsPanel:** `forestChange`, `severityScore`, `renderStats()`
- **SessionState:** `currentMode`, `lastResult`, `resetSession()`

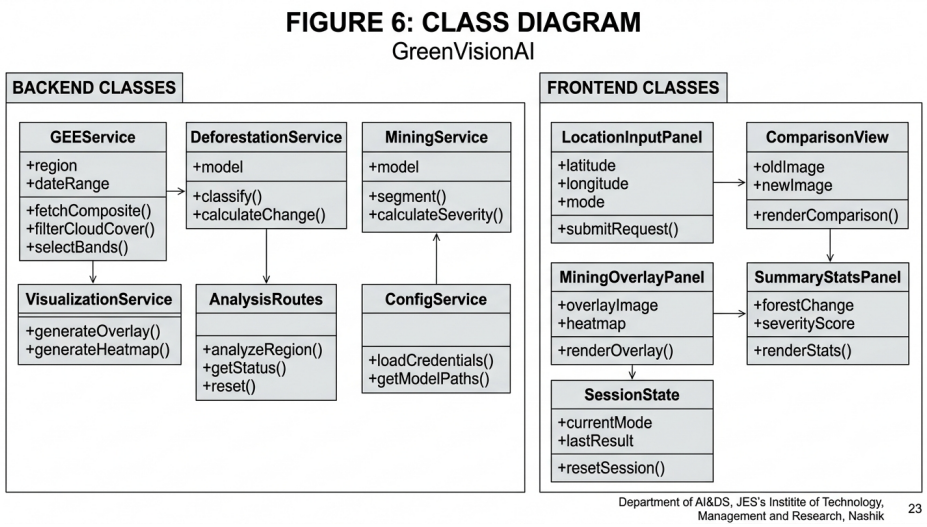


Figure 6: Class Diagram

4.4.3 ACTIVITY DIAGRAM

The activity diagram traces the step-by-step execution from location submission through result display, highlighting decision points and the sequence of actions.

1. User submits coordinates or selects a predefined region.
2. System validates coordinate ranges.
3. System fetches and preprocesses the Sentinel-2 composite via Google Earth Engine.
4. **Decision:** Is the requested analysis mode deforestation or mining?
    - **Deforestation Mode:**
        (a) System runs Random Forest classification on the composite.
        (b) System calculates forest-cover percentage for the historical image.
        (c) System calculates forest-cover percentage for the recent image.
        (d) System computes percentage change and relative loss.
    - **Mining Mode:**
        (a) System runs U-Net segmentation on the composite.

(b) System thresholds the probability mask into a binary overlay.

(c) System computes the severity score from affected area and confidence.

(d) System generates the probability heatmap.

System generates the appropriate visualisation (comparison panel or overlay/heatmap) and returns the result with summary statistics to the user.

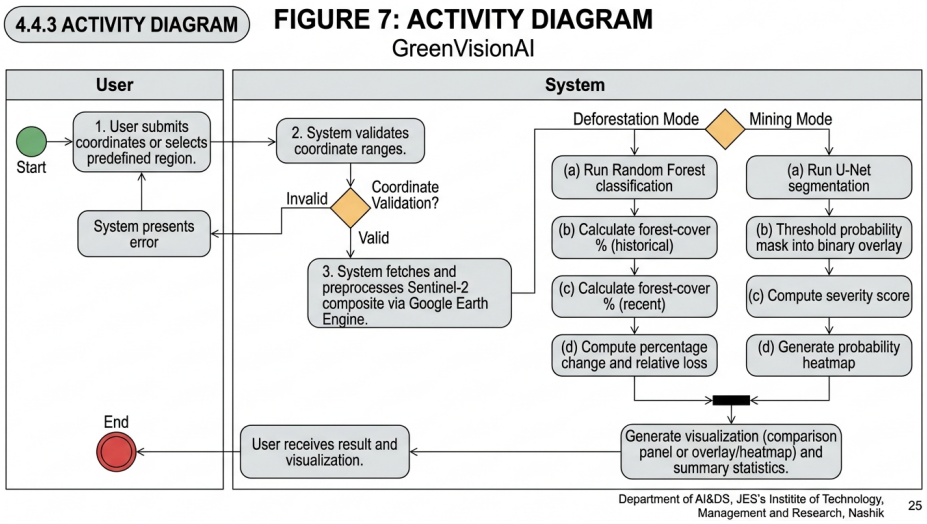


Figure 7: Activity Diagram

4.4.4 SEQUENCE DIAGRAM

The sequence diagram depicts the chronological exchange of messages between system objects during analysis execution.

Participating Objects: User, StreamlitUI, APIClient, FastAPIBackend, GEEService, DeforestationService, MiningService, GoogleEarthEngine.

Sequence (Deforestation Query):

1. User enters coordinates and selects deforestation mode in StreamlitUI.
2. StreamlitUI calls APIClient.analyzeRegion().
3. APIClient sends POST to /api/analyze.
4. FastAPIBackend invokes GEEService.fetchComposite().
5. GEEService queries GoogleEarthEngine for filtered Sentinel-2 imagery.
6. GoogleEarthEngine returns the cloud-free composite.
7. FastAPIBackend invokes DeforestationService.classify().

8. DeforestationService returns forest-cover percentages and change value.
9. FastAPIBackend sends JSON response to StreamlitUI.
10. StreamlitUI displays the comparison panel and statistics to User.

Sequence (Mining Query):

1. User enters coordinates and selects mining mode in StreamlitUI.
2. StreamlitUI calls APIClient.analyzeRegion() with mode=mining.
3. APIClient sends POST to /api/analyze.
4. FastAPIBackend invokes GEEService.fetchComposite().
5. FastAPIBackend invokes MiningService.segment().
6. MiningService returns the probability mask and severity score.
7. FastAPIBackend generates overlay and heatmap via VisualizationService.
8. FastAPIBackend sends JSON response with overlay path to StreamlitUI.
9. StreamlitUI renders the overlay, heatmap, and severity score to User.

Figure 8: Simplified Sequence Diagram

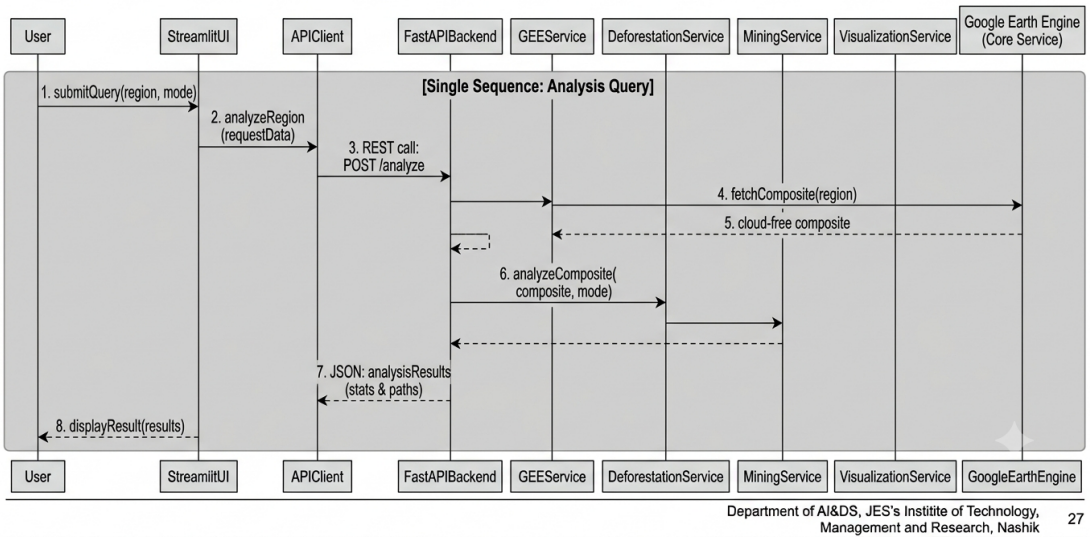


Figure 8: Sequence Diagram

4.4.5 OBJECT DIAGRAM

The object diagram provides a snapshot of the system at runtime when a mining-detection query is processed, showing live object instances with their actual attribute values.

Key Object Instances:

- **user1:User** – Active user interacting with the Streamlit interface.
- **uiInstance:StreamlitUI** – Frontend session with current mode and last result state.
- **apiClient:APIClient** – HTTP client configured with the FastAPI backend base URL.
- **server:FastAPIBackend** – Running Uvicorn server instance on port 8080.
- **geeService:GEEService** – Service instance holding the active region geometry and composite.
- **miningService:MiningService** – Service instance with the loaded U-Net model in evaluation mode.
- **rfModel:RandomForestClassifier** – Trained scikit-learn model instance used for forest classification.

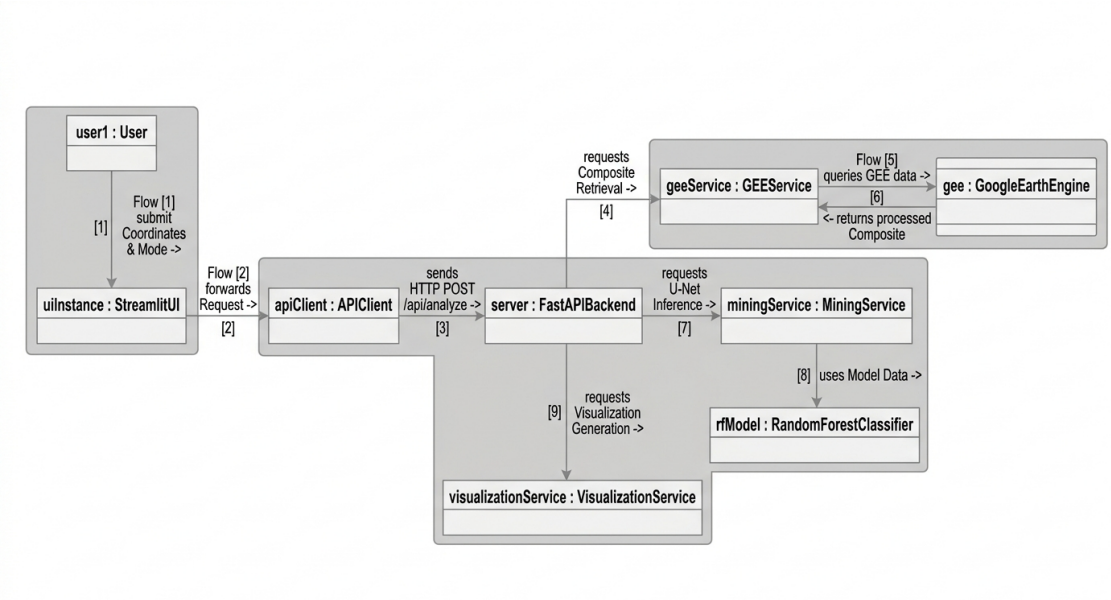


Figure 9: Object Diagram

4.4.6 COLLABORATION DIAGRAM

The collaboration diagram shows the network of objects that collaborate during a mining-detection query, emphasising structural relationships and numbered message flows rather than temporal ordering.

Key Interactions:

1. User submits coordinates through the Streamlit location-input panel.
2. StreamlitUI forwards the request to the APIClient.
3. APIClient sends an HTTP POST to the FastAPI backend (`/api/analyze`).
4. Backend routes the request to GEEService for composite retrieval.
5. GEEService queries Google Earth Engine and returns the processed composite.
6. Backend forwards the composite to MiningService for U-Net inference.
7. MiningService returns the probability mask and severity score.
8. VisualizationService generates the overlay and heatmap images.
9. Final response with statistics and image paths flows back to the Streamlit UI.

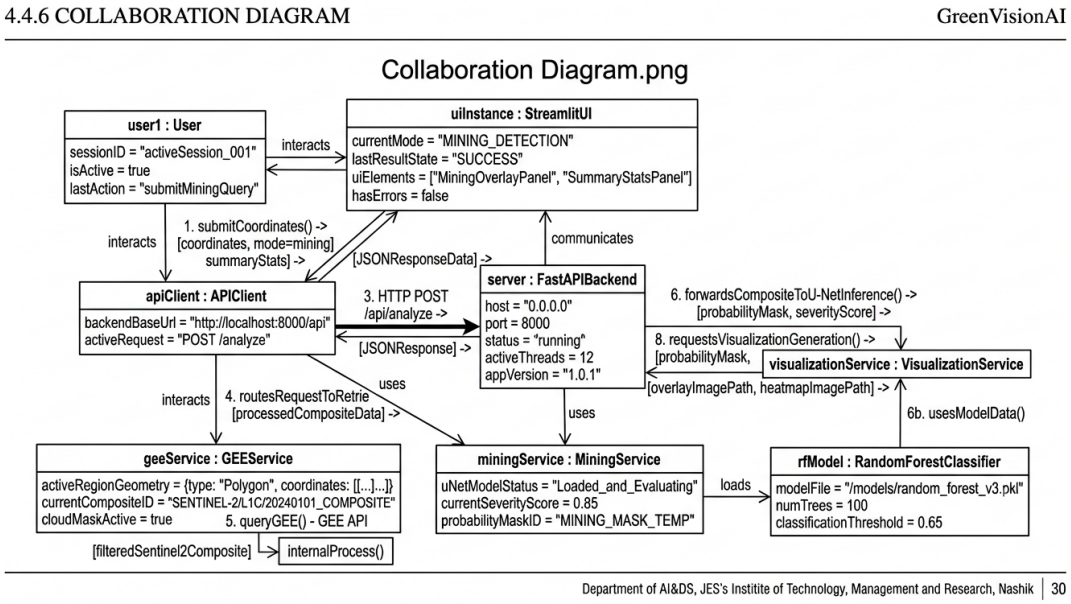


Figure 10: Collaboration Diagram

4.4.7 STATE CHART DIAGRAM

The state chart diagram models the lifecycle of GreenVisionAI from system idle through location submission, satellite retrieval, model inference, and result display. It highlights all transitions and the error-recovery path.

Key States:

- **Idle** – System awaiting a user action.
- **Location Selected** – Coordinates or example region submitted and validated.
- **Fetching Satellite Data** – Google Earth Engine query and composite generation in progress.
- **Processing Deforestation** – Random Forest classification in progress.
- **Processing Mining** – U-Net segmentation in progress.
- **Generating Output** – Overlay, heatmap, and statistics being assembled.
- **Displaying Result** – Final results rendered in the Streamlit interface.
- **Ready** – System awaiting the next analysis request.
- **Error** – API failure or unavailable imagery; system offers retry and returns to Idle.

4.4.7 STATE CHART DIAGRAM GreenVisionAI

FIGURE 11: STATE CHART DIAGRAM State diagram.png

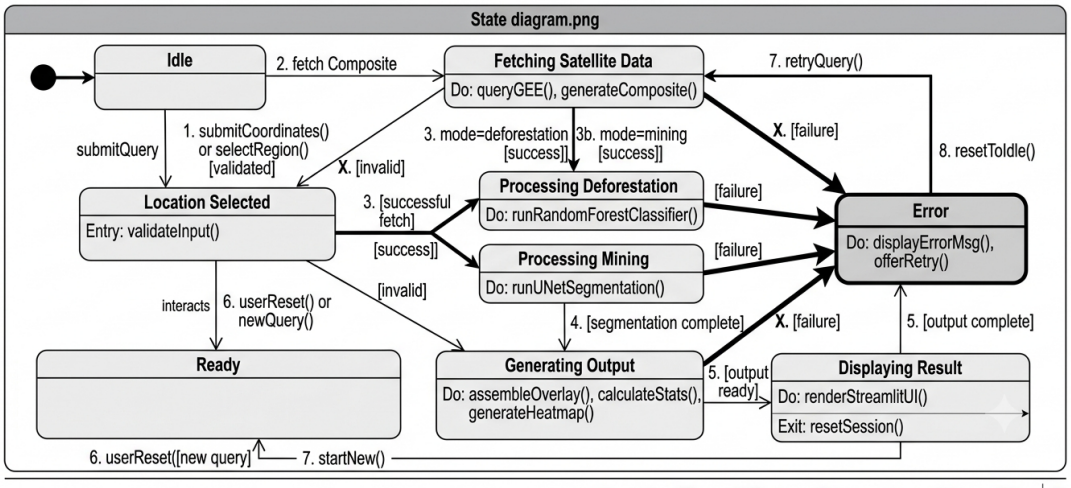


Figure 11: State Chart Diagram

4.4.8 COMPONENT DIAGRAM

The component diagram illustrates the modular structure of GreenVisionAI and the dependency relationships among its major software components across three layers.

Frontend Components:

- **app.py** – Root Streamlit application; mounts input and result panels.
- **LocationInputPanel** – Handles coordinate entry and example-region selection.
- **ComparisonView** – Renders deforestation comparison results.
- **MiningOverlayPanel** – Renders mining overlay and heatmap results.
- **SummaryStatsPanel** – Displays forest change and severity-score statistics.

Backend Services:

- **FastAPI Server** – Receives REST calls from the frontend (port 8080).
- **GEEService** – Earth Engine pipeline: region → filter → composite → band selection.
- **DeforestationService / MiningService** – Load trained models; run classification or segmentation.
- **VisualizationService** – Generate overlays, heatmaps, and severity scores.

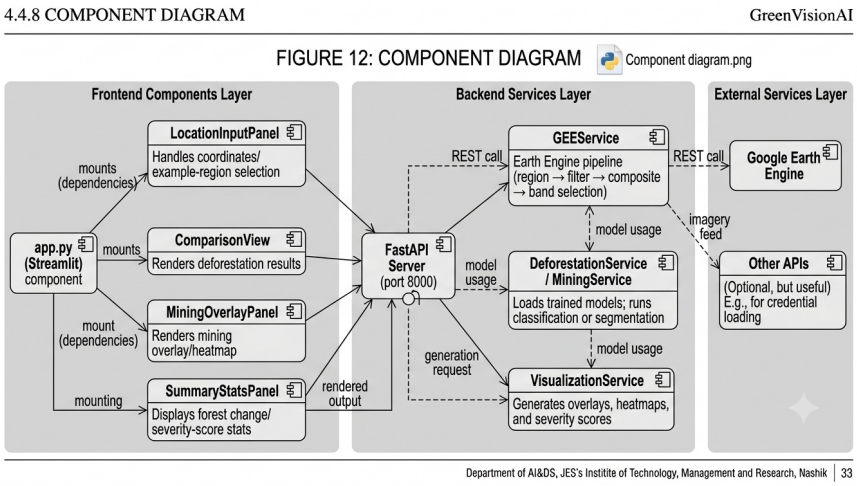


Figure 12: Component Diagram

4.4.9 DEPLOYMENT DIAGRAM

The deployment diagram maps how the software components of GreenVisionAI are distributed across three physical tiers: client device, application server, and external cloud service.[17]

Client Tier (User Device):

- Web browser running the Streamlit application.
- Communicates with the application server over HTTPS.

Application Server (Python):

- FastAPI server (port 8080) hosting GEEService, DeforestationService, and MiningService.
- Locally stored Random Forest (`.pkl`) and U-Net (`.pth`) model files.
- `/outputs` directory for temporary overlay and heatmap storage.

External Cloud Service (Google Earth Engine):

- Sentinel-2 Level-2A image collection for satellite data retrieval.
- Cloud-based filtering and compositing operations.
- Access controlled via a Google service-account credential file.

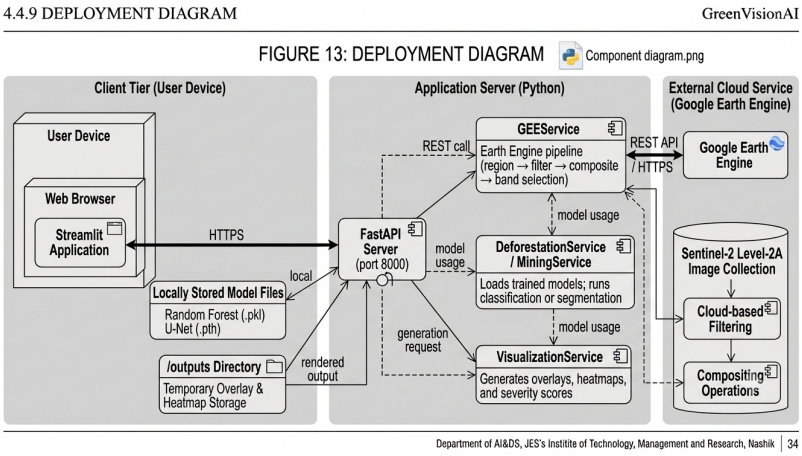


Figure 13: Deployment Diagram

CHAPTER 5: TECHNICAL SPECIFICATIONS

5.1.1 TECHNOLOGY DETAILS USED IN THE PROJECT

The **GreenVisionAI** system is developed using a combination of modern web technologies, geospatial processing tools, and artificial intelligence techniques to create an intelligent, dual-task environmental monitoring platform. The system integrates satellite remote sensing with machine learning and deep learning to deliver accurate, evidence-grounded detection results for deforestation and illegal mining from user-selected geographic regions.

The system follows a three-layer architecture: Frontend (user interface), Backend (application logic), and AI Processing layer.[10]

1. **Frontend Technologies:**
   The frontend is built with **Streamlit** and **Python** for a rapid, component-driven user interface requiring no separate JavaScript tooling. Streamlit's native layout components including `st.columns()`, `st.expander()`, and `st.metric()` render satellite image comparisons, mining overlays, probability heatmaps, and summary statistics for both detection tasks. **Requests** handles HTTP communication with the FastAPI backend, while **Pillow** supports image loading and display within Streamlit components. The interface includes a coordinate input panel with example-region quick-select, a side-by-side satellite comparison view for deforestation results, and a mining overlay panel with severity-score display.[23]

2. **Backend Technologies:**
   The backend is powered by **Python 3.10+** as the runtime environment and **FastAPI** as the web framework, chosen for its asynchronous request handling and native Pydantic-based schema validation. **Uvicorn** serves as the ASGI server hosting the FastAPI application. The backend contains three specialised services: GEEService for satellite data orchestration, DeforestationService for Random Forest based forest-cover analysis, and MiningService for U-Net based mining-region segmentation, each handling a specific processing pipeline independently.[15]

3. **AI and Geospatial Technologies:**
   Google Earth Engine (GEE) provides the geospatial backbone for satellite data acquisition, cloud-cover filtering, and composite generation. **Sentinel-2 Level-2A** imagery is filtered by date range and cloud-cover percentage before a cloud-free

median composite is generated. Selected spectral bands (B2, B3, B4, B8, B11, B12) are extracted and converted into model-ready NumPy arrays for classification and segmentation inference.[12]

4. **Machine Learning and Deep Learning Libraries:**

   - **scikit-learn** – Provides the Random Forest classifier for forest-cover classification and multi-temporal change detection. The trained model is serialised using Joblib and loaded at backend startup.
   - **PyTorch** – Provides the deep learning framework for U-Net convolutional neural network implementation and mining-region segmentation inference.
   - **NumPy** – Supports array operations, band stacking, and pixel-level data manipulation throughout the processing pipeline.
   - **OpenCV (cv2)** – Handles image preprocessing, mask thresholding, overlay compositing, and heatmap colour-mapping for visualisation output.
   - **Joblib** – Serialises and deserialises the trained Random Forest model (`random_forest.pkl`) for efficient loading at backend startup.[21]

5. **Data Visualization:**
   Matplotlib is integrated into the backend to generate colour-mapped probability heatmaps and classified forest-cover output images. When detection completes, the backend saves rendered output images to the `/outputs` directory and returns their file paths to the Streamlit frontend, which renders them inline using `st.image()`. Supported outputs include satellite comparisons, binary mining overlays, probability heatmaps, and severity-score summary panels.[23]

6. **Additional Tools and Technologies:**
   CORS middleware from `fastapi.middleware.cors` is configured to allow cross-origin requests from the Streamlit server. **python-dotenv** manages environment variables including the Google Earth Engine service-account path, model file paths, output directory, and server port. A custom **logger utility** built on Python's `logging` module provides structured request logging and error reporting across all backend services for debugging and monitoring.[24]

The integration of these technologies enables the system to process Sentinel-2 satellite imagery for two environmental detection tasks through a unified interface, deliver classification and segmentation results grounded in real satellite data, and render interactive visual outputs for both deforestation and mining analysis. The modular service architecture allows independent scaling and enhancement of each detection pipeline.[17]

CHAPTER 6: PLANNING PHASE

6.1 PROJECT ESTIMATION

COCOMO Estimation Model

Project estimation plays an important role in software planning because it helps determine the required effort, development duration, and expected cost. For the GreenVisionAI project, the Constructive Cost Model (COCOMO), introduced by Barry Boehm, is used to estimate the development resources.

COCOMO provides a systematic approach for evaluating software projects based on size and development characteristics. The three categories of the COCOMO model are:

1. **Basic COCOMO**
   This model estimates software effort primarily using project size measured in KLOC (Thousand Lines of Code).
   $$E = a \times (KLOC)^b$$
   where E = Estimated effort in person-months, a, b = project-specific constants, KLOC = estimated size of software.

2. **Intermediate COCOMO**
   This model improves estimation accuracy by incorporating additional project-related factors such as team capability, hardware limitations, software complexity, and development environment.

3. **Advanced COCOMO**
   This model provides a detailed estimation by considering software reuse, reliability requirements, project distribution, and operational conditions.

Development Category

The GreenVisionAI project falls under the **Semi-Detached Development Mode**. This category is suitable for projects that combine familiar web development components with specialised machine learning and geospatial processing modules requiring moderate team experience.

Table 1: Project Development Mode

Mode	Project Size	Complexity	Timeline	Environment
Semi-Detached	Medium	Moderate-High	Moderate	Mixed

6.1.1 Estimation Formula

Effort estimation is calculated using:

$$E = a \times (KLOC)^b$$

For Semi-Detached projects: $a = 3.0$, $b = 1.12$

6.1.2 Project Calculation

Effort Calculation

The GreenVisionAI system consists of a FastAPI backend with three core services, a Streamlit frontend with multiple result panels, Google Earth Engine integration, trained machine learning and deep learning models, and configuration modules. The estimated size is **8.00 KLOC**.

$$E = 3.0 \times (8.00)^{1.12}$$

$$E \approx 30.80 \text{ Person-Months}$$

Estimated Duration

Project duration is calculated using:

$$D = 2.5 \times E^{0.38}$$

$$D = 2.5 \times (30.80)^{0.38} \approx 8.3 \approx 6 \text{ Months}$$

The estimated development period for GreenVisionAI is approximately six months.

Required Team Size

The team size is estimated as:

$$N = \frac{E}{D} = \frac{30.80}{8.3} \approx 4 \text{ members}$$

Therefore, four team members are considered sufficient for project execution.

Estimated Cost

The total project cost is determined using:

$$C = N \times C_p = 4 \times 7415 = 29660$$

The approximate implementation cost is estimated to be Rs. 29660.

6.2 PROJECT SCHEDULE AND TEAM ORGANIZATION

The development process is divided into structured stages to ensure systematic execution and timely completion.

Table 2: Development Activities

Task	Activity
T1	Topic Selection
T2	Requirement Analysis
T3	Technology Exploration (Google Earth Engine, Random Forest, U-Net CNN)
T4	Environment Setup (Python, FastAPI, Streamlit, GEE credentials)
T5	Satellite Data Pipeline Development (Sentinel-2 retrieval, compositing)
T6	Deforestation Module Development (Random Forest training and integration)
T7	Mining Detection Module Development (U-Net training and integration)
T8	Visualization Development (overlays, heatmaps, severity scores)
T9	Frontend Development (Streamlit UI, result panels)
T10	Backend API Development (FastAPI routes, services)
T11	System Integration and Testing
T12	Documentation
T13	Deployment
T14	Maintenance

Team Structure

Table 3: GreenVisionAI Team Members

Developer ID	Team Member
D1	Barkare Vedika Nandkishor
D2	Patil Pranjali Mohan
D3	Patil Shubham Dnyaneshwar
D4	Sonawane Pravin Rajesh

CHAPTER 7: SOFTWARE IMPLEMENTATION

7.1 INTRODUCTION

Software implementation converts the system design into a working application. GreenVisionAI is implemented as a modular web application with separate frontend, backend, and AI processing components. The implementation focuses on two distinct processing pipelines: a Random Forest classification pipeline for deforestation detection and a U-Net segmentation pipeline for illegal mining detection, both driven by Sentinel-2 satellite imagery acquired through Google Earth Engine.

Main implementation features include:

1. **Satellite Data Ingestion:** Users submit geographic coordinates or select a predefined example region through the Streamlit interface. The backend constructs a region-of-interest geometry, queries the Sentinel-2 Level-2A collection on Google Earth Engine, applies cloud-cover filtering, and generates a cloud-free median composite ready for model inference.

2. **Deforestation Detection Pipeline:** The processed composite is passed to the Random Forest classifier, which assigns each pixel a land-cover label. Forest-cover percentages are calculated for historical and recent composites and compared to quantify forest loss or gain over the selected time period.

3. **Mining Detection Pipeline:** The U-Net convolutional neural network receives the satellite composite and produces a pixel-wise probability mask. The mask is thresholded into a binary overlay, and a severity score is computed from the affected area and model confidence values.

4. **Result Visualisation:** The FastAPI backend returns JSON responses containing forest-cover statistics, mining overlays, probability heatmaps, and severity scores. The Streamlit frontend renders these outputs through interactive panels, side-by-side satellite comparisons, and summary statistics displays.

7.2 PROJECT STRUCTURE

Listing 1: ForestVision Project Structure

```
greenvision/
|-- backend/
|   |-- __pycache__/
|   |-- model_ressources/
|   |-- __init__.py
|   |-- custom_layer.py
|   |-- fix_model.py
|   |-- gee_image_loader.py
|   |-- image_array_loader.py
|   |-- main.py
|   |-- mining_detector.py
|   |-- segmenter.py
|   |-- sentinelhub_requester.py
|   |-- utils.py
|-- frontend/
|   |-- .streamlit/
|   |-- images/
|   |-- pages/
|   |   |-- 2_Mining_Detection.py
|   |   |-- 3_Chatbot_Assistant.py
|   |-- utils/
|   |-- GreenVision.py
|-- venv/
|-- .gitignore
|-- .python-version
|-- Dockerfile
|-- README.md
|-- package-lock.json
|-- requirements.txt
|-- setup.py
```

7.3 SATELLITE DATA PROCESSING PIPELINE

The satellite data processing pipeline forms the core data acquisition layer of GreenVisionAI. It begins with region selection, where the backend constructs a region-of-interest geometry from user-submitted coordinates. Sentinel-2 Level-2A imagery is then queried from Google Earth Engine for the specified date range and filtered by cloud-cover

percentage to ensure image quality. A cloud-free median composite is generated from the filtered collection, and relevant spectral bands (B2, B3, B4, B8, B11, B12) are selected and exported as a NumPy array for downstream classification and segmentation inference.

Listing 2: Satellite Data Acquisition and Composite Generation

```
import ee
import numpy as np

def fetch_sentinel_composite(latitude, longitude, start_date, end_date):
    region = ee.Geometry.Point([longitude, latitude]).buffer(5000)
    collection = (
        ee.ImageCollection("COPERNICUS/S2_SR")
        .filterBounds(region)
        .filterDate(start_date, end_date)
        .filter(ee.Filter.lt("CLOUDY_PIXEL_PERCENTAGE", 10))
    )
    composite = collection.median().select(
        ["B2", "B3", "B4", "B8", "B11", "B12"]
    )
    band_data = composite.sampleRectangle(region=region, defaultValue=0)
    array = np.stack([
        np.array(band_data.get(b).getInfo())
        for b in ["B2", "B3", "B4", "B8", "B11", "B12"]
    ], axis=-1)
    return array
```

7.4 ENVIRONMENT CONFIGURATION

The backend environment requires Python dependencies including FastAPI, Uvicorn, earthengine-api, NumPy, scikit-learn, PyTorch, and OpenCV. The Streamlit frontend requires Python and the Streamlit package along with the Requests and Pillow libraries. Google Earth Engine authentication must be configured using a service-account credential file before any satellite image requests can be processed. All sensitive configuration values are stored in environment variables managed through a `.env` file loaded by `python-dotenv`.

Listing 3: Environment Variables and Configuration

```
# Server Configuration
PORT=8080
HOST=0.0.0.0
ENV=development
```

```
5
6  # Google Earth Engine
7  GEE_SERVICE_ACCOUNT=your-service-account@project.iam.gserviceaccount.com
8  GEE_KEY_FILE=path/to/service-account-key.json
9
10 # Model Paths
11 RF_MODEL_PATH=backend/models/random_forest.pkl
12 UNET_MODEL_PATH=backend/models/unet_model.pth
13
14 # Output Directory
15 OUTPUT_DIR=outputs/
16
17 # Processing Configuration
18 MAX_CLOUD_COVER=10
19 DEFAULT_BUFFER_METERS=5000
20 UNET_THRESHOLD=0.5
21
22 # Logging
23 LOG_LEVEL=info
24 LOG_DIR=logs/
```

7.5 BUSINESS LOGIC

The business logic of GreenVisionAI is divided into two main workflows: deforestation detection and illegal mining detection. In the deforestation workflow, the system retrieves historical and recent Sentinel-2 composites for the selected region. The Random Forest classifier assigns land-cover labels to each pixel and calculates forest-cover percentage for both time periods. The difference between the two values is returned as the forest loss or gain metric, giving environmental analysts a direct, quantitative measure of vegetation change.

In the mining detection workflow, the U-Net CNN model receives the processed satellite composite and produces a pixel-wise probability mask. Pixels exceeding the configured probability threshold are flagged as mining-affected. The system overlays the binary mask on the satellite image and computes a severity score based on the proportion of affected pixels and the mean model confidence across flagged regions. Both workflows expose their results through FastAPI endpoints that return structured JSON responses consumed by the Streamlit frontend.

Listing 4: FastAPI Analysis Endpoint

```python
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from services.gee_service import fetch_sentinel_composite
from services.deforestation_service import classify_forest_cover
from services.mining_service import segment_mining_regions

router = APIRouter()

class AnalysisRequest(BaseModel):
    latitude: float
    longitude: float
    start_date: str
    end_date: str
    mode: str  # "deforestation" or "mining"

@router.post("/api/analyze")
async def analyze_region(request: AnalysisRequest):
    try:
        composite = fetch_sentinel_composite(
            request.latitude, request.longitude,
            request.start_date, request.end_date
        )
        if request.mode == "deforestation":
            result = classify_forest_cover(composite)
        elif request.mode == "mining":
            result = segment_mining_regions(composite)
        else:
            raise HTTPException(status_code=400, detail="Invalid mode")
        return {"success": True, "result": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Listing 5: Forest Cover Change Calculation

```python
def calculate_forest_change(old_forest_percent, new_forest_percent):
    absolute_change = new_forest_percent - old_forest_percent
    relative_loss = (
        (old_forest_percent - new_forest_percent) / old_forest_percent
    ) * 100
    return {
        "old_forest_percent": old_forest_percent,
        "new_forest_percent": new_forest_percent,
```

```
        "absolute_change": round(absolute_change, 2),
        "relative_loss": round(relative_loss, 2)
    }
```

API Endpoints:

Table 4: GreenVisionAI API Endpoints

Method	**Endpoint**	**Description**
GET	/	Returns API status and model information
GET	/get_satellite_images	Fetches Sentinel-2 imagery via GEE and runs Random Forest classification for two dates
GET	/get_interval_images	Fetches multiple satellite images across a date range for interval-based deforestation analysis
GET	/detect_mining	Fetches satellite image for a single date and runs U-Net CNN mining detection

The core processing logic ensures that analysis requests are routed to the correct pipeline based on the mode parameter. Both pipelines invoke their respective trained models with the preprocessed Sentinel-2 composite as input. All API responses follow a consistent JSON structure containing `success`, `result`, and optional `overlay_path` or `heatmap_path` fields for image-based outputs. Processing status is communicated to the Streamlit frontend through status indicators updated at each pipeline stage.

CHAPTER 8: SOFTWARE TESTING

8.1 INTRODUCTION

Software testing is a critical phase in the development of GreenVisionAI that verifies whether the system performs according to its specified requirements. Testing ensures that each component of the application operates correctly, reliably, and consistently under various input conditions. For GreenVisionAI, testing covers coordinate input validation, satellite data acquisition, deforestation detection accuracy, mining segmentation output, API response verification, and user interface rendering. A structured test plan was followed to validate both functional and non-functional requirements across all major system components.

Testing is organized at the module level, covering location input validation, Random Forest based deforestation analysis, and U-Net based mining detection, followed by complete system-level integration testing. This structured approach improves reliability, confirms correct API behavior, and ensures that the application handles both valid inputs and edge cases gracefully.

8.1.2 TEST CASE FOR LOCATION INPUT

The location input test verifies whether the system correctly accepts valid latitude and longitude coordinate values within the permissible range and appropriately rejects invalid or out-of-range values with clear error messages. This ensures that downstream satellite data acquisition is only triggered for geographically valid regions.

Table 5: Test Cases for Location Input

TC ID	Test Case	Input	Expected Output	Status
TC-L1	Valid coordinates	Lat: 23.75, Lon: 86.42	Region accepted	Pass
TC-L2	Invalid latitude	Lat: 200, Lon: 86.42	Error: out of range	Pass
TC-L3	Invalid longitude	Lat: 23.75, Lon: 250	Error: out of range	Pass
TC-L4	Empty fields	No input submitted	Validation message	Pass
TC-L5	Predefined region	Bolivia Sugarcane selected	Coordinates auto-filled	Pass
TC-L6	Non-numeric input	Lat: abc, Lon: xyz	Error: invalid format	Pass

8.1.3 TEST CASE FOR DEFORESTATION DETECTION

The deforestation detection test verifies whether the Random Forest classifier correctly generates forest cover percentages for historical and recent Sentinel-2 composites and accurately identifies the magnitude of forest loss or gain between the two selected time periods. The test also validates that the side-by-side satellite comparison and forest cover change statistics are correctly rendered in the Streamlit interface.

Table 6: Test Cases for Deforestation Detection

TC ID	Test Case	Input	Expected Output	Status
TC-D1	Valid deforestation query	Coordinates with date range 2017–2024	Forest cover change returned	Pass
TC-D2	Historical composite	Start date: 2017-01-01	Cloud-free composite generated	Pass
TC-D3	Recent composite	End date: 2024-12-31	Cloud-free composite generated	Pass
TC-D4	Forest classification	Processed composite input	Land cover labels returned	Pass
TC-D5	Change detection	Old and new forest percentages	Change computed correctly	Pass
TC-D6	No imagery available	Region with no Sentinel-2 data	Graceful error displayed	Pass

8.1.4 TEST CASE FOR MINING DETECTION

The mining detection test verifies whether the U-Net CNN model correctly produces pixel-wise probability masks, binary mining overlays, probability heatmaps, and severity scores for a selected region. The test also validates that the generated overlay images are correctly saved to the outputs directory and returned to the Streamlit frontend for display.

Table 7: Test Cases for Mining Detection

TC ID	Test Case	Input	Expected Output	Status
TC-M1	Valid mining query	Jharia Coal Mine coordinates	Overlay and heatmap generated	Pass
TC-M2	Probability mask	Processed composite input	Pixel-wise mask returned	Pass
TC-M3	Binary overlay	Probability above threshold	Overlay image saved	Pass
TC-M4	Severity score	Affected area and confidence	Score computed correctly	Pass
TC-M5	Heatmap generation	Probability mask input	Colour-mapped heatmap generated	Pass
TC-M6	Low mining region	Minimal mining area	Low severity, small overlay	Pass

8.2 SNAPSHOT OF TEST CASE

The following observations were recorded during integration testing of the GreenVisionAI system:

- Location input module correctly validates coordinate ranges and rejects invalid or out-of-range values with appropriate error messages.
- Satellite data acquisition through Google Earth Engine successfully retrieves and composites Sentinel-2 imagery within the expected processing time for typical study regions.
- Random Forest deforestation pipeline correctly classifies forest cover and computes change percentages between historical and recent composites.
- U-Net mining detection pipeline produces accurate probability masks, binary overlays, and severity scores for known mining regions including the Jharia Coal Mine, Jharkhand.
- API endpoints return correctly structured JSON responses for both deforestation and mining analysis modes.
- Streamlit interface correctly renders satellite comparisons, mining overlays, heatmaps, and summary statistics panels for all tested regions.

CHAPTER 9: RESULT

9.1 SNAPSHOTS OF THE RESULTS

The result phase demonstrates the output of GreenVisionAI for selected study regions. The system displays visual and quantitative results for both deforestation detection and illegal mining detection.

DEFORESTATION DETECTION

The deforestation module compares satellite imagery between two selected years. In the camera-ready experiment, the system compared imagery from 2017 and 2024. The output indicated a major reduction in forest cover, showing change from 83 percent forest cover in 2017 to 25 percent forest cover in 2024, resulting in an approximate total decrease of 70 percent.

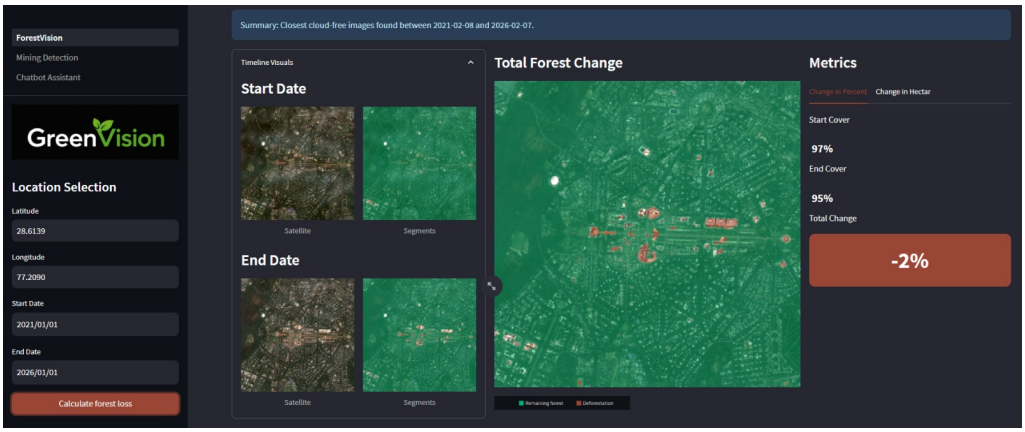


Figure 14: Deforestation Detection Output



Figure 15: Forest Cover Change Panel

ILLEGAL MINING DETECTION

The illegal mining module was demonstrated on the Jharia Coal Mine region in Jharkhand. The U-Net CNN model generated mining detection overlays and probability heatmaps. Mining regions were highlighted in red to make them visually identifiable.

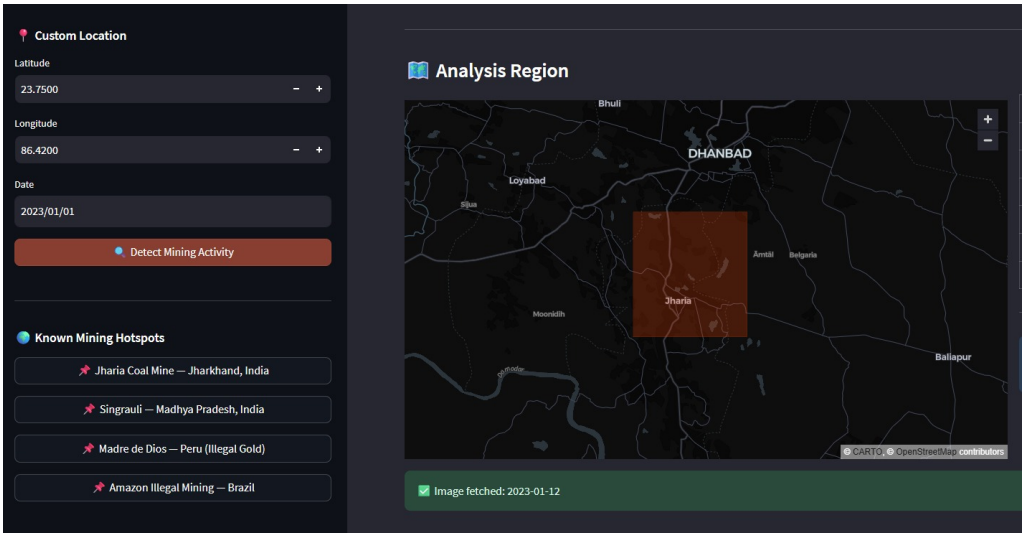


Figure 16: Illegal Mining Detection Output

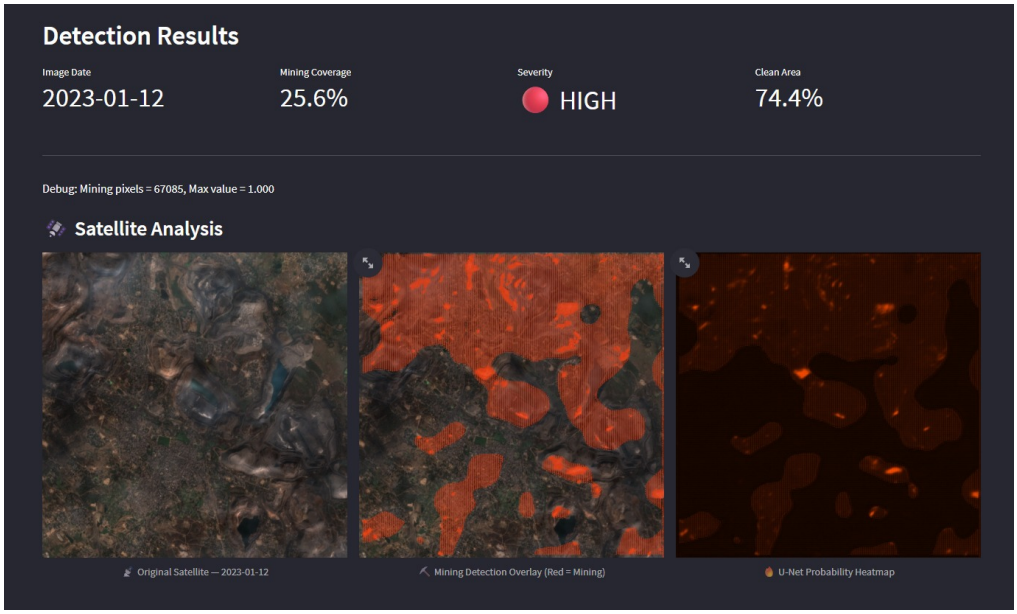


Figure 17: Mining Heatmap and Severity Score

9.2 OBSERVATION AND EVALUATION

The experimental results show that GreenVisionAI can identify significant forest cover changes and probable mining affected regions from satellite imagery. The Random Forest model achieved strong performance for forest cover classification, while the U-Net model produced useful segmentation outputs for mining detection.

Table 8: Comparative Performance of Detection Methods

Method	**Task**	**Accuracy / IoU**	**Remarks**
Random Forest	Forest Cover Classification	93.8% Accuracy	Proposed
U-Net CNN	Mining Region Segmentation	91.3% IoU	Proposed
FCN without skip connection	Mining Segmentation	83.1% IoU	Baseline
RF Pixel Classifier	Mining Segmentation	78.4% IoU	Baseline
NDVI Thresholding	Deforestation Detection	80.0% Accuracy	Baseline
Manual Survey	Field Inspection	Variable	Slow and costly

The results demonstrate the value of combining satellite remote sensing with machine learning and deep learning. The system can reduce manual effort and support faster identification of environmental degradation.

CHAPTER 10: CONCLUSIONS AND FUTURE WORK

10.1 CONCLUSIONS

GreenVisionAI successfully demonstrates an AI-based framework for automated detection of deforestation and illegal mining using Sentinel-2 satellite imagery accessed through Google Earth Engine. The system integrates a Random Forest classifier for forest cover mapping and temporal change detection with a U-Net Convolutional Neural Network for pixel-wise mining region segmentation, delivering dual-task environmental monitoring capabilities within a single unified web-based platform. The FastAPI backend services and Streamlit frontend interface provide an accessible and interactive experience that requires no specialist GIS expertise from the user.

Key achievements of the project include:

- Implementation of a complete satellite data processing pipeline using Google Earth Engine for Sentinel-2 Level-2A image acquisition, cloud-cover filtering, and cloud-free composite generation.
- Development of a Random Forest based forest cover classification module with multi-temporal change detection achieving 93.8 percent overall classification accuracy.
- Development of a U-Net CNN based mining region segmentation module achieving 91.3 percent mean IoU, significantly outperforming conventional baseline methods.
- Integration of dual-task deforestation and illegal mining detection within a single application, addressing the limitation of existing single-task monitoring tools.
- Experimental validation demonstrating forest cover reduction from 83 percent to 25 percent over the 2017 to 2024 period and successful identification of illegal mining activity at the Jharia Coal Mine region in Jharkhand.
- Development of a responsive Streamlit web interface with side-by-side satellite comparison panels, mining overlays, probability heatmaps, and severity score displays accessible from any modern browser.
- Clean separation of concerns through a modular, service-oriented FastAPI backend architecture enabling independent scaling and enhancement of each detection pipeline.

The results confirm that combining satellite remote sensing with machine learning and deep learning provides an effective, scalable, and cost-effective alternative to traditional field-based environmental monitoring. The system is capable of analysing large geographical regions automatically, reducing manual effort, and supporting faster identification of ecological degradation. The proposed framework provides a practical solution for environmental surveillance, sustainable forest governance, and rapid regulatory decision-making by government agencies and environmental authorities.

10.2 FUTURE WORK

Building on the present work, the following targeted improvements and extensions are planned for future development of the GreenVisionAI system:

- **Real-Time Alert Generation:** Integrate automated alerting for sudden vegetation loss or mining expansion events, enabling near-real-time notification to environmental monitoring agencies and government authorities.
- **SAR Data Fusion:** Incorporate Sentinel-1 Synthetic Aperture Radar imagery alongside Sentinel-2 optical data to improve detection accuracy during monsoon seasons when cloud cover limits optical image availability.
- **Additional Satellite Datasets:** Integrate complementary satellite sources such as Landsat, MODIS, and high-resolution commercial imagery to improve spatial detail and temporal coverage for environmental analysis.
- **Expanded Training Dataset:** Improve model generalization by training on larger and more geographically diverse labelled datasets covering different forest types, mining patterns, and climatic regions.
- **Active Learning Integration:** Incorporate active learning strategies to iteratively improve model accuracy by selectively labelling the most informative satellite image patches during deployment.
- **Mobile Application Support:** Develop native mobile applications for iOS and Android to enable field officers and environmental inspectors to access monitoring outputs directly from mobile devices.
- **Automated PDF Report Generation:** Add functionality to generate downloadable PDF reports summarising analysis results, forest cover statistics, mining severity scores, and satellite image comparisons for selected regions.

- **Role-Based Dashboards:** Provide customised dashboards for different user roles including environmental analysts, government officers, and administrative authorities with access controls and exportable summaries.
- **Temporal Animation:** Integrate multi-year temporal animation to visualise land cover change progression over several years, providing a more intuitive representation of long-term environmental degradation trends.
- **Field Verification Integration:** Incorporate field verification data and ground truth samples to continuously validate and improve model reliability and detection confidence across diverse geographical regions.

Future work should also focus on tightening the retrieval-to-inference integration within the satellite processing pipeline, refining severity scoring mechanisms for more granular environmental impact assessment, and developing systematic evaluation and monitoring strategies for production deployments to maintain reliability and user trust over time.

REFERENCES

[1] Pan Y, Birdsey RA, Fang J, et al. A Large and Persistent Carbon Sink in the World's Forests. Science. 2011;333(6045):988–993.

[2] Potapov P, Hansen MC, Laestadius L, et al. The Last Frontiers of Wilderness: Tracking Loss of Intact Forest Landscapes from 2000 to 2013. Science Advances. 2017;3(1):e1600821.

[3] Drusch M, Del Bello U, Carlier S, et al. Sentinel-2: ESA's Optical High-Resolution Mission for GMES Operational Services. Remote Sensing of Environment. 2012;120:25–36.

[4] Tucker CJ, Townshend JRG, Goff TE. African Land-Cover Classification Using Satellite Data. Science. 1985;227(4685):369–375.

[5] Hansen MC, Potapov PV, Moore R, et al. High-Resolution Global Maps of 21st-Century Forest Cover Change. Science. 2013;342(6160):850–853.

[6] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation. In: Proceedings of MICCAI 2015. Cham: Springer; 2015. p. 234–241.

[7] Bischke B, Bhatt P, Breuel T, et al. Multi-Task Learning for Segmentation of Building Footprints from Satellite Imagery. Proceedings of IEEE ICIP. 2019;1480–1484.

[8] Ghaffarian S, Ghaffarian S. Automatic Building Detection Based on Supervised Classification Using High-Resolution Google Earth Images. International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences. 2014;XL-1:101–106.

[9] Gorelick N, Hancher M, Dixon M, et al. Google Earth Engine: Planetary-Scale Geospatial Analysis for Everyone. Remote Sensing of Environment. 2017;202:18–27.

[10] Hansen MC, Krylov A, Tyukavina A, et al. Humid Tropical Forest Disturbance Alerts Using Landsat Data. Environmental Research Letters. 2016;11(3):034008.

[11] Shimada M, Itoh T, Motooka T, et al. New Global Forest/Non-Forest Maps from ALOS PALSAR Data (2007–2010). Remote Sensing of Environment. 2014;155:13–31.

[12] Breiman L. Random Forests. Machine Learning. 2001;45(1):5–32.

[13] He K, Zhang X, Ren S, Sun J. Deep Residual Learning for Image Recognition. In: Proceedings of IEEE CVPR. 2016. p. 770–778.

[14] Zhu XX, Tuia D, Mou L, et al. Deep Learning in Remote Sensing: A Comprehensive Review and List of Resources. IEEE Geoscience and Remote Sensing Magazine. 2017;5(4):8–36.

[15] Maxwell AE, Warner TA, Fang F. Implementation of Machine-Learning Classification in Remote Sensing: An Applied Review. International Journal of Remote Sensing. 2018;39(9):2784–2817.

[16] Sommerville I. Software Engineering. 10th ed. Pearson; 2016.

[17] Pressman RS, Maxim BR. Software Engineering: A Practitioner's Approach. 9th ed. McGraw-Hill; 2020.

GLOSSARY

Term	Description
Artificial Intelligence	A branch of computer science that enables machines to perform tasks that typically require human-level intelligence, such as pattern recognition, decision making, and image analysis.
Band Composite	A combination of selected spectral bands from a satellite image used as input to machine learning models for land cover classification and segmentation tasks.
Cloud Cover	The portion of a satellite image obscured by clouds, which hides land surface details and must be filtered before analysis to ensure composite quality.
Convolutional Neural Network	A class of deep learning models designed for processing structured grid data such as images, using convolutional layers to extract spatial features automatically.
Deep Learning	A subfield of machine learning that uses neural networks with multiple layers to learn hierarchical representations from data for tasks such as image classification and segmentation.
Deforestation	The removal or significant reduction of forest cover due to human activities such as agriculture, industrial expansion, urban development, or illegal resource extraction.
Encoder-Decoder Architecture	A neural network design pattern in which an encoder compresses input into a latent representation and a decoder reconstructs the output, commonly used in image segmentation models such as U-Net.
FastAPI	A modern, high-performance Python web framework for building RESTful APIs with asynchronous request handling and native Pydantic-based schema validation.
Google Earth Engine	A cloud-based geospatial processing platform developed by Google that provides access to large-scale satellite imagery archives and computational infrastructure for environmental analysis.

Illegal Mining	Mining activity carried out without legal authorization or outside the boundaries of approved operational limits, often causing severe ecological damage to surrounding land and water resources.
Intersection over Union	A standard evaluation metric for image segmentation models that measures the overlap between the predicted segmentation mask and the ground truth mask as a ratio of their intersection to their union.
Land Cover Classification	The process of assigning each pixel in a satellite image to a predefined land cover category such as dense forest, degraded forest, bare soil, water body, or built-up area.
NDVI	Normalized Difference Vegetation Index — a spectral index computed from the red and near-infrared bands of a satellite image used to quantify vegetation density and health.
Random Forest	An ensemble machine learning algorithm that constructs multiple decision trees during training and combines their predictions through majority voting to produce robust classification outputs.
Region of Interest	A user-defined geographic area represented as a geometry object used to spatially filter satellite image collections and restrict analysis to the selected location.
Remote Sensing	The science of collecting information about the Earth's surface from a distance using satellite or airborne sensors without physical contact with the observed area.
Sentinel-2	A European Space Agency satellite mission providing free, high-resolution multispectral imagery with a five-day revisit cycle, widely used for vegetation monitoring and land cover analysis.
Severity Score	A quantitative metric computed from the proportion of mining-affected pixels and the mean model confidence across flagged regions, used to prioritise follow-up field inspections.
Streamlit	An open-source Python framework for building interactive web applications and data visualisation dashboards without requiring separate frontend development tools.
U-Net	A convolutional neural network architecture originally developed for biomedical image segmentation, featuring an encoder-decoder structure with skip connections that preserve fine spatial detail for pixel-wise classification tasks.

ANNEXURE A
LITERATURE REVIEW TABLE

Sr. No.	Author(s) & Year	Title / Methodology	Key Findings & Relevance to Proposed System
1	Tucker et al. (1985)	*African Land-Cover Classification Using Satellite Data.* Demonstrated the effectiveness of NDVI derived from Landsat imagery for identifying vegetation and tropical deforestation patterns.	Established NDVI as a foundational spectral index for vegetation monitoring; the proposed system extends this with machine learning-based classification using multi-band Sentinel-2 composites.
2	Hansen et al. (2013)	*High-Resolution Global Maps of 21st-Century Forest Cover Change.* Developed one of the first globally consistent forest loss mapping systems using Landsat time-series and decision-tree classifiers.	Provided the benchmark framework for large-scale forest change detection; the proposed Random Forest classifier builds on this approach with higher-dimensional spectral features and a unified dual-task architecture.
3	Drusch et al. (2012)	*Sentinel-2: ESA's Optical High-Resolution Mission for GMES Operational Services.* Described the Sentinel-2 satellite mission, its multispectral band configuration, and its 10-metre spatial resolution.	Sentinel-2 Level-2A imagery forms the primary data source for GreenVisionAI; its free accessibility and five-day revisit cycle make it highly suitable for near-real-time environmental monitoring.

Continued on next page

continued from previous page

Sr. No.	Author(s) & Year	Title / Methodology	Key Findings & Relevance to Proposed System
4	Ronneberger et al. (2015)	*U-Net: Convolutional Networks for Biomedical Image Segmentation.* Introduced the encoder-decoder architecture with skip connections for pixel-wise segmentation of biomedical images.	The U-Net architecture is directly adopted in GreenVisionAI for pixel-wise mining region segmentation; skip connections preserve fine spatial boundaries critical for accurate mining footprint detection.
5	Gorelick et al. (2017)	*Google Earth Engine: Planetary-Scale Geospatial Analysis for Everyone.* Presented the Google Earth Engine platform for cloud-based geospatial data processing and satellite image analysis.	Google Earth Engine serves as the geospatial backbone of GreenVisionAI, enabling large-scale Sentinel-2 image acquisition, cloud-cover filtering, and composite generation without local data download.
6	Ghaffarian et al. (2014)	*Automatic Building Detection Based on Supervised Classification Using High-Resolution Google Earth Images.* Applied CNN-based classification to identify land features from high-resolution imagery with accuracy above 90%.	Demonstrated that deep learning models can accurately detect irregular land patterns from satellite imagery; motivates the use of U-Net CNN for mining footprint detection in GreenVisionAI.

Continued on next page

continued from previous page

Sr. No.	Author(s) & Year	Title / Methodology	Key Findings & Relevance to Proposed System
7	Bischke et al. (2019)	*Multi-Task Learning for Segmentation of Building Footprints from Satellite Imagery.* Explored encoder-decoder segmentation models adapted from biomedical imaging to remote sensing applications.	Confirmed the effectiveness of U-Net based architectures for remote sensing segmentation tasks; supports the adaptation of U-Net for mining region detection in the proposed system.
8	Hansen et al. (2016)	*Humid Tropical Forest Disturbance Alerts Using Landsat Data.* Demonstrated near-real-time deforestation alert generation using satellite imagery and machine learning at planetary scale through the GLAD alert system.	Established the feasibility of near-real-time forest monitoring at scale; GreenVisionAI extends this concept by combining deforestation and mining detection in a single interactive web platform.
9	Breiman (2001)	*Random Forests.* Introduced the Random Forest ensemble algorithm combining multiple decision trees for robust classification with high accuracy on high-dimensional datasets.	Random Forest is directly used in GreenVisionAI for forest cover classification across six land-cover classes using an 18-dimensional spectral and textural feature vector per pixel.

ANNEXURE B
CODE SNIPPETS

B.1 RANDOM FOREST BASED FOREST CLASSIFICATION

```
def extract_features(img_array):
r, g, b, nir = [img_array[:, :, i].astype(np.float32) for i in range(4)]
eps = 1e-8
ndvi = (nir - r) / (nir + r + eps)
ndwi = (g - nir) / (g + nir + eps)
evi = 2.5 * (nir - r) / (nir + 6*r - 7.5*b + 1 + eps)
savi = 1.5 * (nir - r) / (nir + r + 0.5 + eps)
brightness = (r + g + b + nir) / 4
features = np.stack([ndvi, ndwi, evi, savi, r, g, b, nir, brightness],
    ↪ axis=2)
return features.reshape(-1, features.shape[2])
def generate_training_labels(img_array):
r, nir = img_array[:, :, 0].astype(np.float32), img_array[:, :, 3].astype(
    ↪ np.float32)
ndvi = (nir - r) / (nir + r + 1e-8)
labels = np.full(ndvi.shape, -1, dtype=np.int8) # -1 = uncertain
labels[ndvi > 0.45] = 1 # confident forest
labels[ndvi < 0.15] = 0 # confident non-forest
return labels.flatten()
def train_model(img_array):
features = extract_features(img_array)
labels = generate_training_labels(img_array)
mask = labels != -1 # use only confident pixels
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features[mask])
clf = RandomForestClassifier(
n_estimators=100, max_depth=10,
n_jobs=-1, random_state=42, class_weight='balanced'
)
clf.fit(X_scaled, labels[mask])
return clf, scaler
def segment(img_array, model=None, threshold=0.3):
clf, scaler = load_or_train_model(img_array)
```

```
X = scaler.transform(extract_features(img_array))
predictions = clf.predict(X)
mask = predictions.reshape(512, 512).astype(np.float32)
mask = (uniform_filter(mask, size=3) > 0.5).astype(np.float32)
return mask
```

B.2 U-NET MINING SEGMENTATION INFERENCE

```
class ConvBlock(nn.Module):
def __init__(self, in_ch, out_ch):
super().__init__()
self.block = nn.Sequential(
nn.Conv2d(in_ch, out_ch, 3, padding=1), nn.BatchNorm2d(out_ch),
nn.ReLU(inplace=True),
nn.Conv2d(out_ch, out_ch, 3, padding=1), nn.BatchNorm2d(out_ch),
nn.ReLU(inplace=True)
)
def forward(self, x): return self.block(x)
class MiningUNet(nn.Module):
def __init__(self):
super().__init__()
self.enc1, self.enc2, self.enc3 = ConvBlock(6,16), ConvBlock(16,32),
    ↪ ConvBlock(32,64)
self.pool = nn.MaxPool2d(2)
self.bottleneck = ConvBlock(64, 128)
self.up3, self.dec3 = nn.ConvTranspose2d(128,64,2,stride=2), ConvBlock
    ↪ (128,64)
self.up2, self.dec2 = nn.ConvTranspose2d(64,32,2,stride=2), ConvBlock
    ↪ (64,32)
self.up1, self.dec1 = nn.ConvTranspose2d(32,16,2,stride=2), ConvBlock
    ↪ (32,16)
self.out, self.sigmoid = nn.Conv2d(16,1,1), nn.Sigmoid()
def forward(self, x):
e1 = self.enc1(x)
e2 = self.enc2(self.pool(e1))
e3 = self.enc3(self.pool(e2))
b = self.bottleneck(self.pool(e3))
d3 = self.dec3(torch.cat([self.up3(b), e3], dim=1))
d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))
```

```
return self.sigmoid(self.out(d1))
def detect_mining(img_array, threshold=0.4):
model = load_or_train_unet(img_array)
model.eval()
features = extract_mining_features(img_array) # (6, 512, 512)
x = torch.tensor(features).unsqueeze(0) # (1, 6, 512, 512)
with torch.no_grad():
prob_map = model(x).squeeze().numpy() # (512, 512)
mining_mask = (prob_map > threshold).astype(np.float32)
return mining_mask, prob_map
```

B.3 FASTAPI ANALYSIS ENDPOINT

```
def numpy_to_base64(img_array):
buf = io.BytesIO()
np.save(buf, img_array)
buf.seek(0)
return base64.b64encode(buf.read()).decode("utf-8")
@app.get("/get_satellite_images")
def get_satellite_images(
start_timeframe: str, end_timeframe: str,
longitude: str, latitude: str,
send_orginal_images: str = "False"
):
date_list = [start_timeframe, end_timeframe]
loaded_dates, img_arrays = load_images_from_satellite(
lat_deg=float(latitude), lon_deg=float(longitude), date_list=date_list
)
segmented = [segment(img, threshold=0.3) for img in img_arrays]
return safe_response(
dates=loaded_dates,
segmented=[numpy_to_base64(s) for s in segmented]
)
@app.get("/detect_mining")
def detect_mining_endpoint(latitude: str, longitude: str, date: str):
loaded_dates, img_arrays = load_images_from_satellite(
lat_deg=float(latitude), lon_deg=float(longitude), date_list=[date]
)
img = img_arrays[0]
mining_mask, prob_map = detect_mining(img, threshold=0.4)
```

```
mining_pct = float(mining_mask.sum() / mining_mask.size * 100)
return JSONResponse(content={
"actual_date": loaded_dates[0],
"mining_percentage": round(mining_pct, 2),
"mining_mask": numpy_to_base64(mining_mask),
"prob_map": numpy_to_base64(prob_map.astype(np.float32)),
})
```

B.4 FOREST COVER CHANGE CALCULATION

```
HECTAR_PER_IMAGE = 2621.44
def calculate_metrics(dates, segmented_images):
coverage = [100 - (np.count_nonzero(arr == 0) / arr.size * 100)
for arr in segmented_images]
df = (
pd.DataFrame({"date": dates, "coverage": coverage})
.assign(prev_coverage = lambda d: d.coverage.shift(1))
.assign(cover_diff_pp = lambda d: d.coverage - d.prev_coverage)
.assign(cover_diff_ha = lambda d: d.cover_diff_pp / 100 * HECTAR_PER_IMAGE
    ↪ )
.assign(cover_diff_ha_cum = lambda d: d.cover_diff_ha.cumsum())
.assign(loss_per_months_ha= lambda d: d.cover_diff_ha / d.
    ↪ months_since_prev)
)
return df
# In ForestVision.py  triggered on button click
def process_calculation_request(latitude, longitude, start_date, end_date)
    ↪ :
response = request_satellite_images(url=API_URL, latitude=latitude,
longitude=longitude, start_date=start_date.strftime("%Y-%m-%d"),
end_date=end_date.strftime("%Y-%m-%d"))
image_dates, segmented_images, raw_images = parse_response(response)
metrics_df = calculate_metrics(image_dates, segmented_images).reset_index
    ↪ ()
cover_start = metrics_df.at[0, "coverage"]
cover_end = metrics_df.at[1, "coverage"]
cover_diff_ha = convert_to_ha(cover_end - cover_start)
```

ANNEXURE C
USER MANUAL

C.1 SYSTEM OVERVIEW

GreenVisionAI is a web-based AI-powered environmental monitoring platform that enables users to detect deforestation and illegal mining activity for any geographic region using real-time Sentinel-2 satellite imagery. The system is designed to provide accurate, evidence-grounded environmental analysis through an accessible Streamlit interface backed by a FastAPI service layer, requiring no specialist GIS software installation or expertise from the end user.

C.1.1 Key Features

- **Dual-Task Detection:** Combined deforestation and illegal mining detection within a single unified web application.
- **Satellite Data Integration:** Automated Sentinel-2 Level-2A image acquisition and cloud-free composite generation through Google Earth Engine.
- **Random Forest Classification:** Forest cover mapping and multi-temporal change detection achieving 93.8 percent classification accuracy.
- **U-Net Segmentation:** Pixel-wise mining region detection with probability heatmaps and severity scoring achieving 91.3 percent mean IoU.
- **Interactive Interface:** Streamlit-based web interface with real-time processing feedback, satellite comparison panels, and mining overlay displays.

C.2 SYSTEM REQUIREMENTS

C.2.1 Hardware Requirements

- Laptop or desktop computer with a modern web browser.
- Minimum 8 GB RAM (16 GB recommended for U-Net inference).
- Stable internet connection for Google Earth Engine API access.

- GPU optional for faster U-Net inference; CPU sufficient for demonstration.

C.2.2 Software Prerequisites

- Python 3.10 or higher.
- Google Earth Engine account with service-account credentials configured.
- FastAPI and Uvicorn for backend service hosting.
- Streamlit for frontend interface.
- scikit-learn, PyTorch, NumPy, and OpenCV for model inference.

C.3 INSTALLATION AND SETUP

C.3.1 Step 1: Install Backend Dependencies

```
cd backend
pip install -r requirements.txt
```

C.3.2 Step 2: Configure Google Earth Engine

```
earthengine authenticate
# Or configure service account credentials in .env file
GEE_SERVICE_ACCOUNT=your-service-account@project.iam.gserviceaccount.com
GEE_KEY_FILE=path/to/service-account-key.json
```

C.3.3 Step 3: Install Frontend Dependencies

```
cd frontend
pip install streamlit requests pillow
```

C.3.4 Step 4: Launch the Application

```
# Start FastAPI backend
uvicorn backend.app:app --host 0.0.0.0 --port 8080

# Start Streamlit frontend
streamlit run frontend/app.py
```

C.4 USER OPERATIONS

C.4.1 Select Location

Enter latitude and longitude coordinates directly in the input fields or select a predefined example region from the quick-select dropdown. The system supports any valid geographic coordinates and includes example regions such as Bolivia Sugarcane, Brazil Soy, and Jharia Coal Mine for demonstration purposes.

C.4.2 Run Deforestation Detection

Select the deforestation analysis mode and specify the historical and recent date ranges. Submit the request to trigger Sentinel-2 image acquisition and Random Forest classification. The system displays a side-by-side satellite comparison and forest cover percentage statistics for both time periods along with the absolute and relative change values.

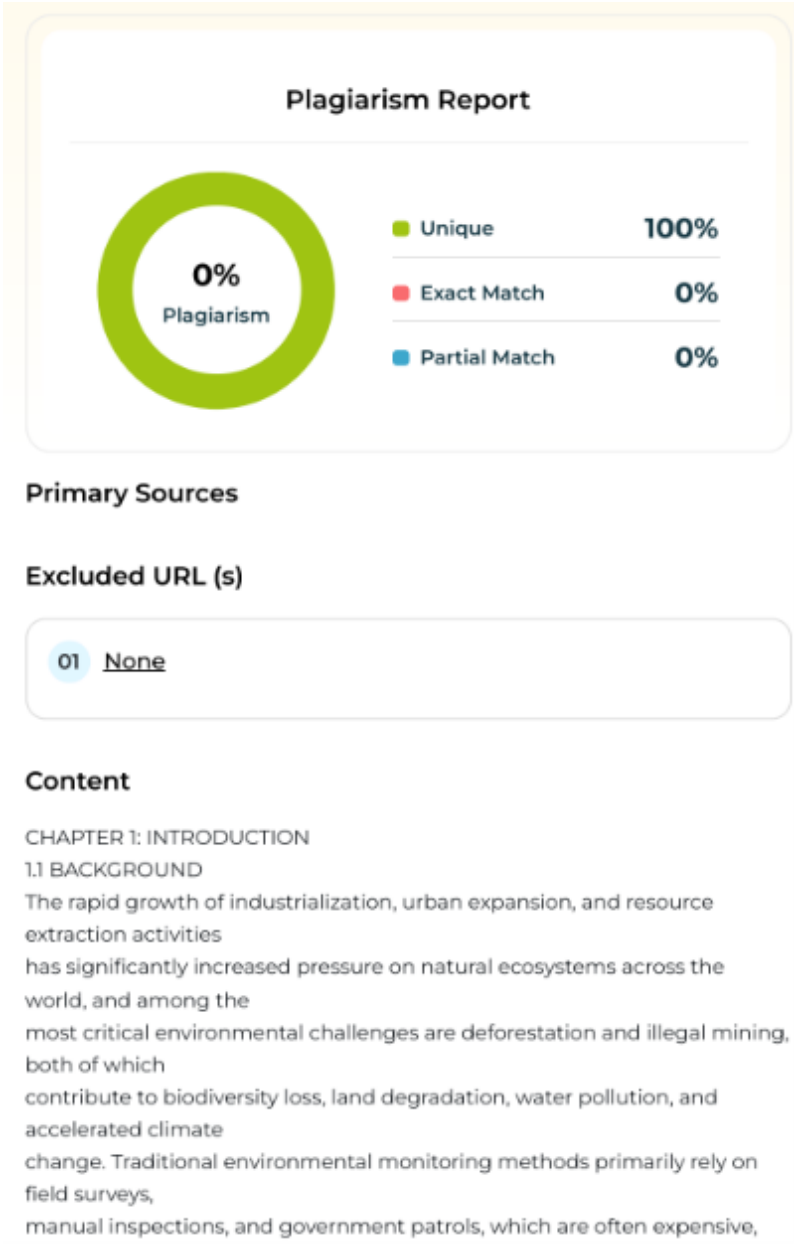
C.4.3 Run Illegal Mining Detection

Select the mining detection mode and submit the request with the desired coordinates and date range. The U-Net model processes the satellite composite and generates a binary mining overlay, probability heatmap, and severity score. Mining-affected regions are highlighted in red in the overlay panel for clear visual identification.

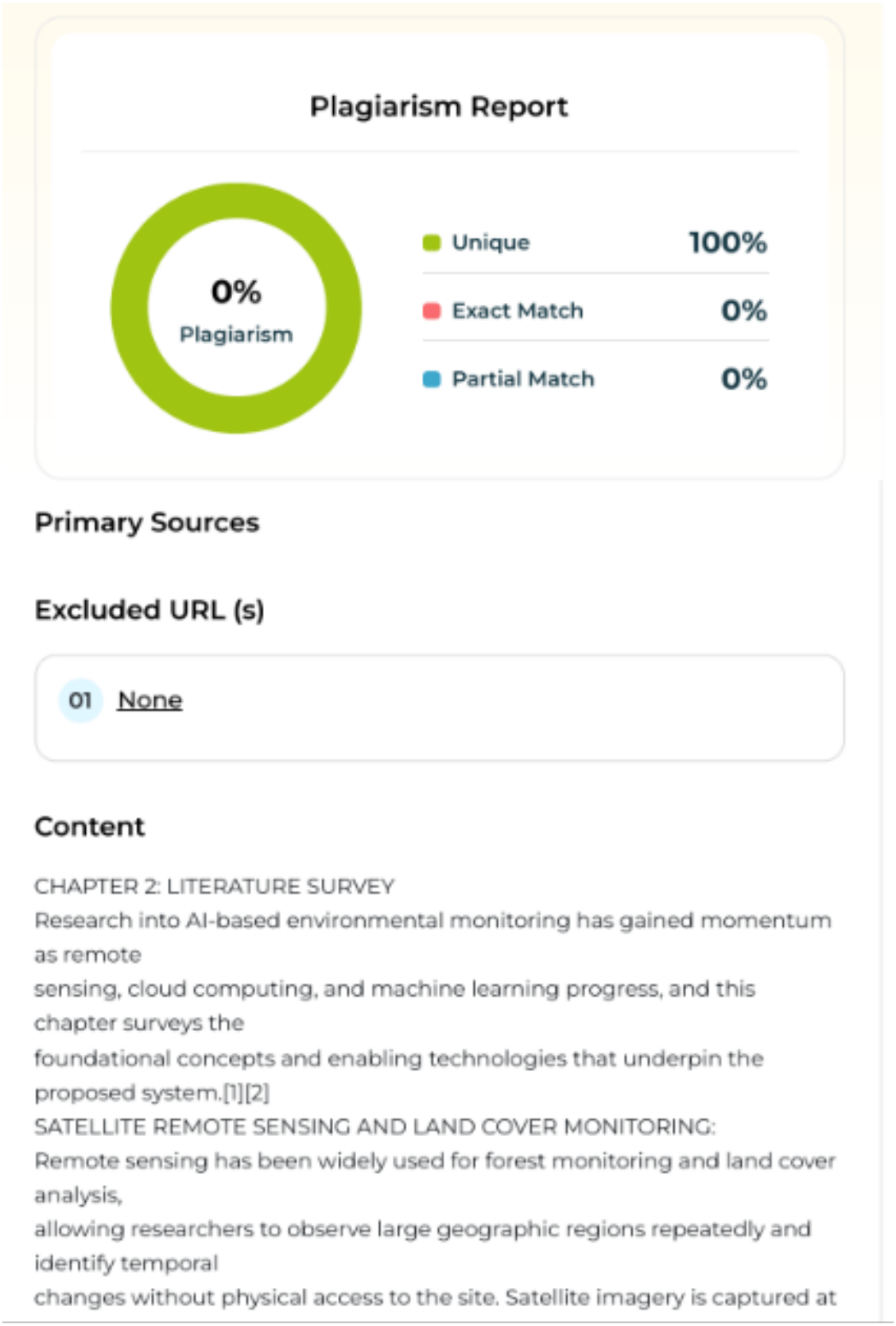
C.4.4 View Results

Observe satellite image comparisons, classified forest cover outputs, mining overlays, probability heatmaps, and summary statistics from the result panels in the Streamlit interface. All outputs are generated from real Sentinel-2 satellite data and grounded in model predictions for the selected geographic region.

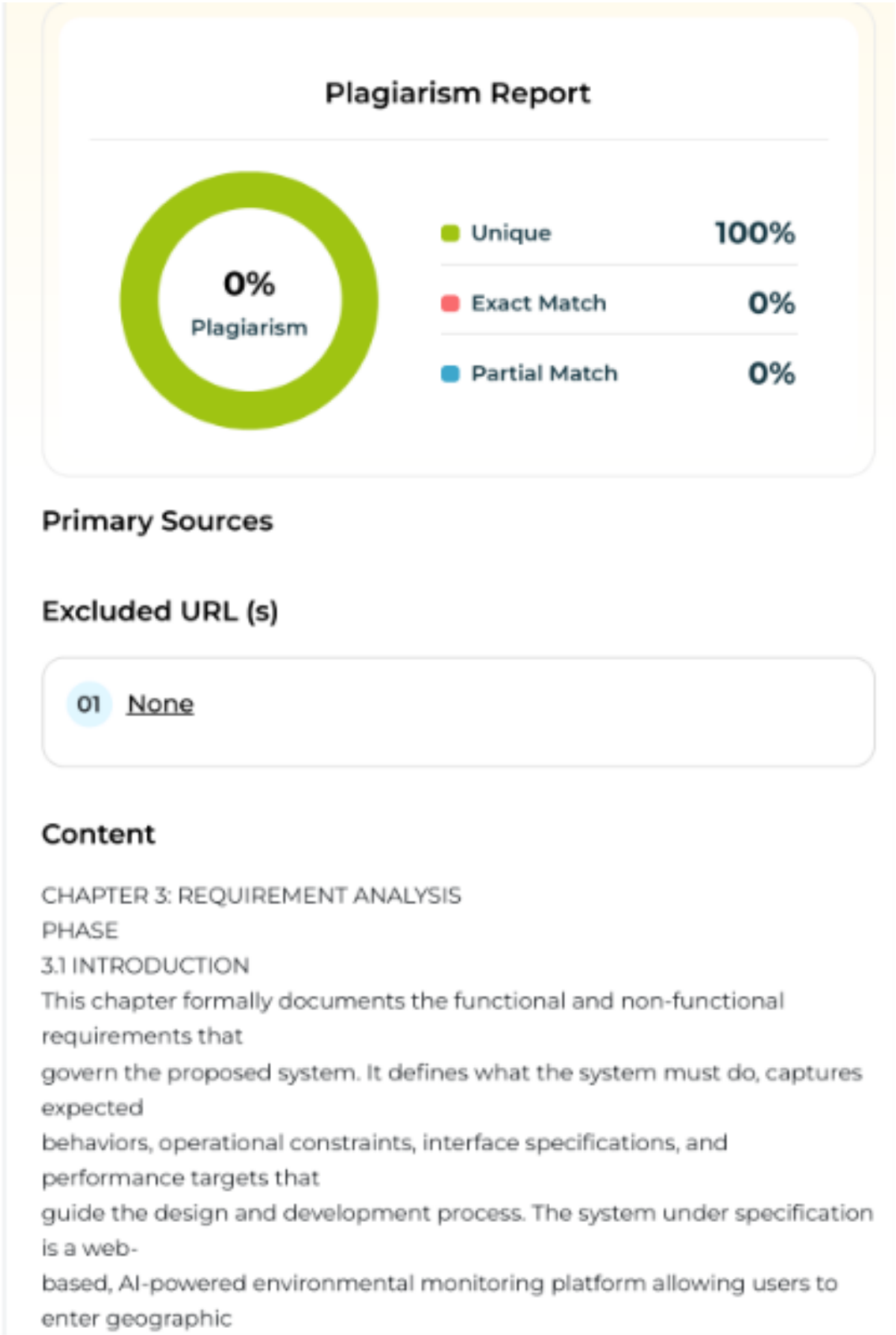
ANNEXURE D
PLAGIARISM REPORT



Plagiarism Report for Chapter 1



Plagiarism Report for Chapter 2



Plagiarism Report

0%
Plagiarism

Unique 100%
Exact Match 0%
Partial Match 0%

Primary Sources

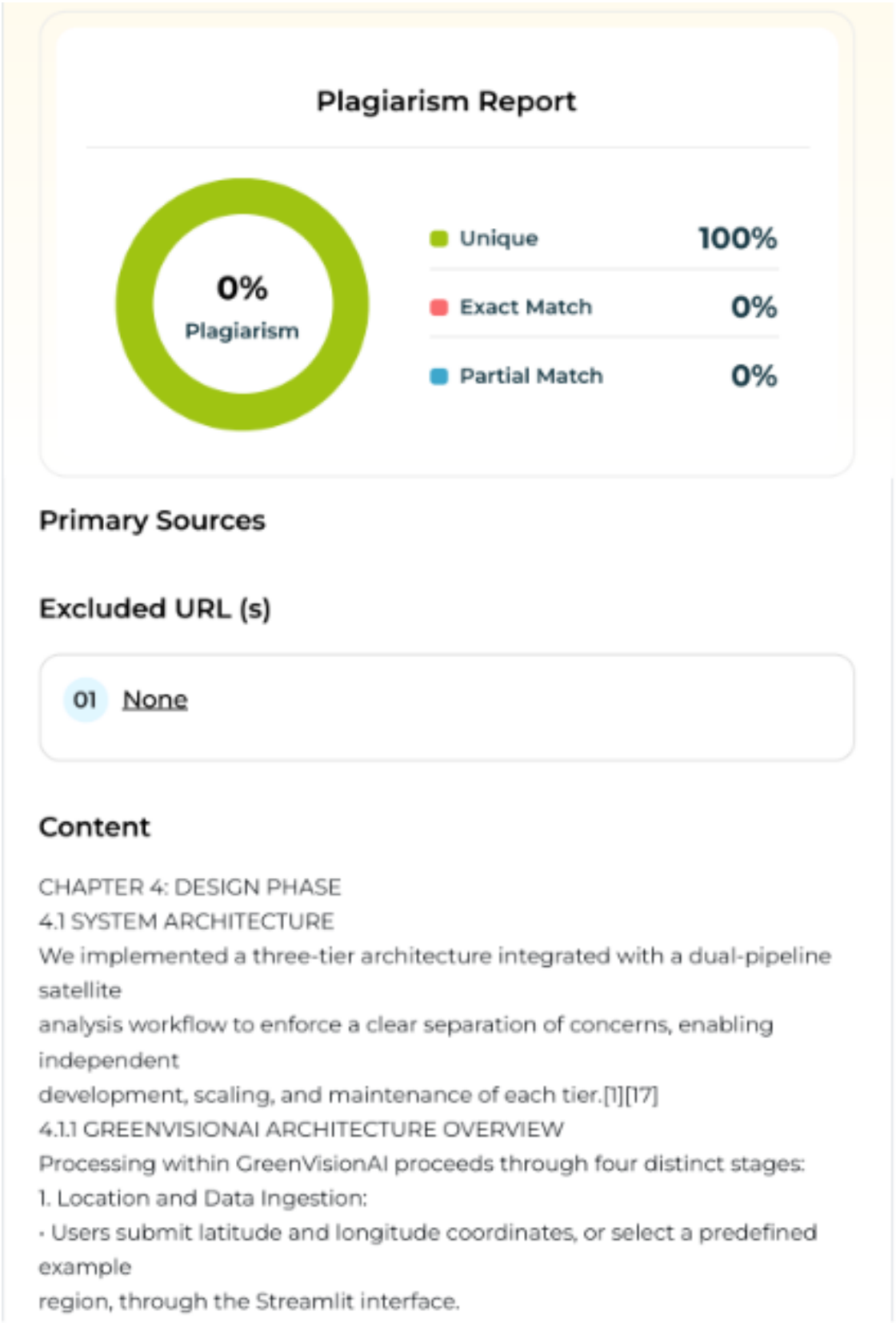
Excluded URL (s)

01 None

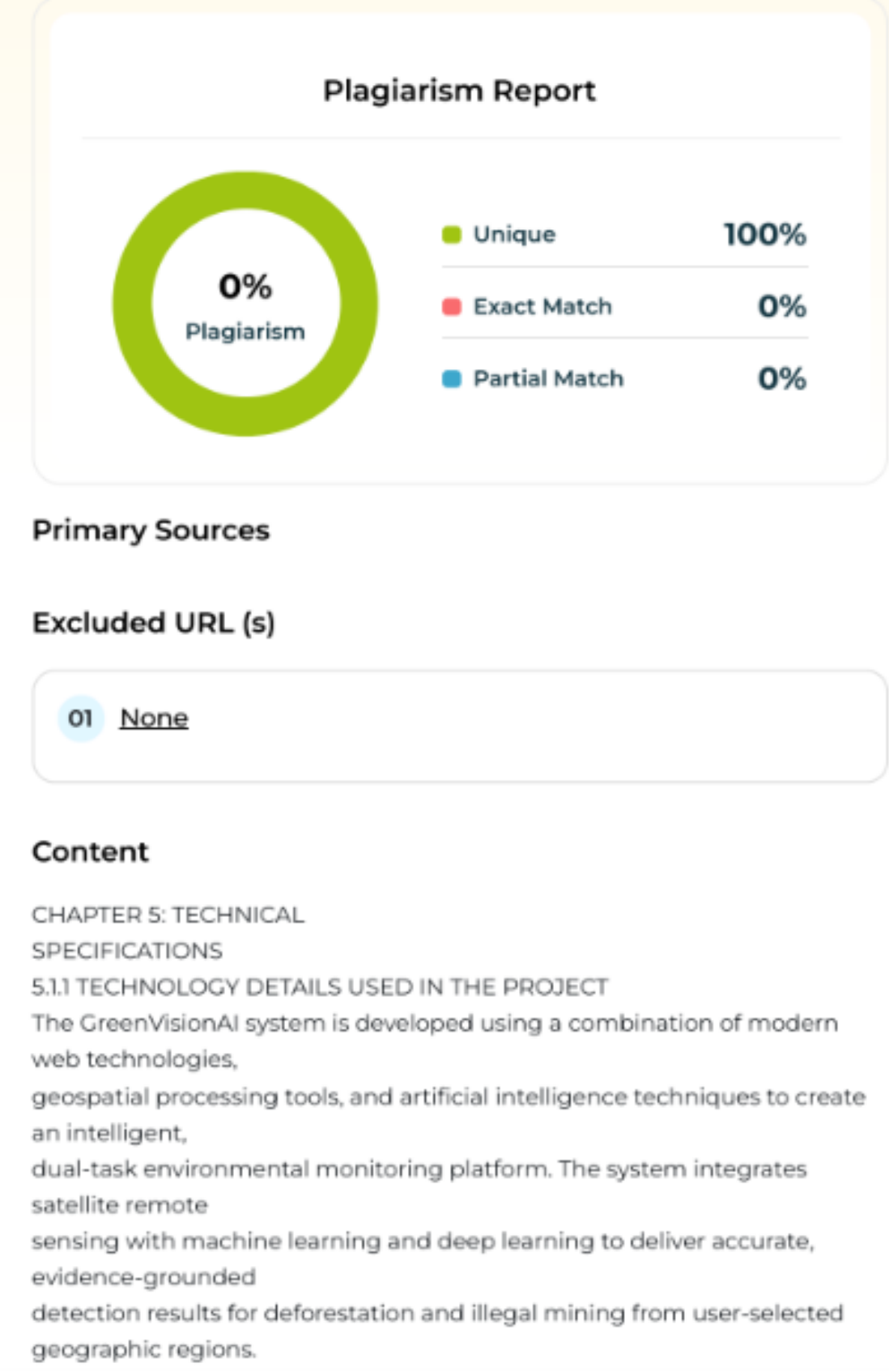
Content

CHAPTER 3: REQUIREMENT ANALYSIS
PHASE
3.1 INTRODUCTION
This chapter formally documents the functional and non-functional requirements that
govern the proposed system. It defines what the system must do, captures expected
behaviors, operational constraints, interface specifications, and performance targets that
guide the design and development process. The system under specification is a web-
based, AI-powered environmental monitoring platform allowing users to enter geographic

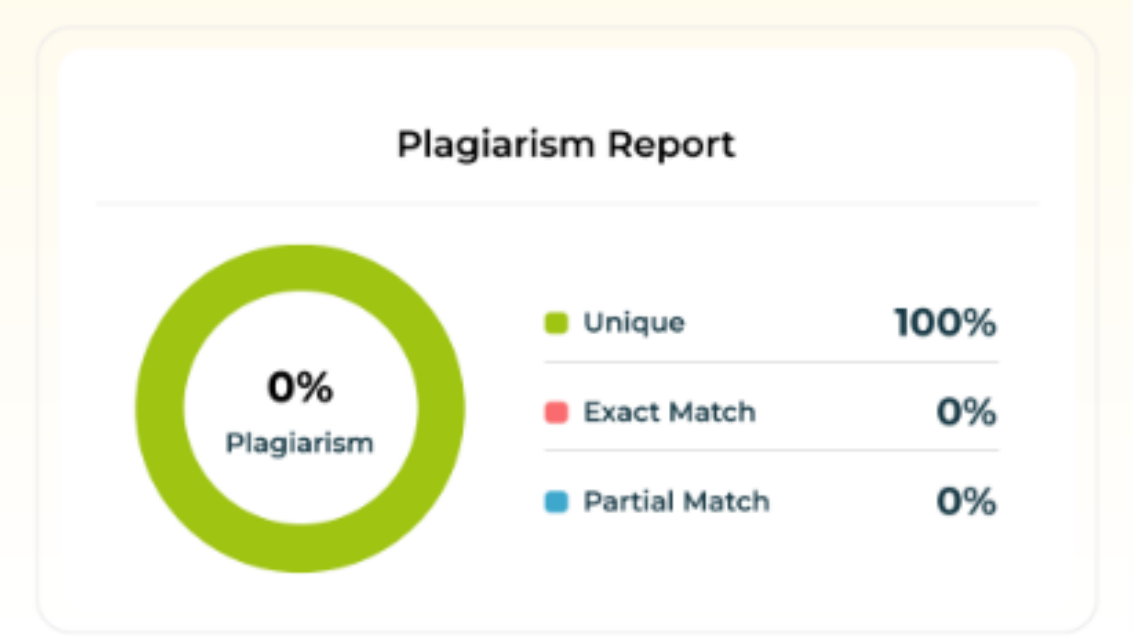
Plagiarism Report for Chapter 3



Plagiarism Report for Chapter 4



Plagiarism Report for Chapter 5



Primary Sources

Excluded URL (s)

01 None

Content

CHAPTER 6: PLANNING PHASE
6.1 PROJECT ESTIMATION
COCOMO Estimation Model
Project estimation plays an important role in software planning because it helps determine
the required effort, development duration, and expected cost. For the GreenVisionAI
project, the Constructive Cost Model (COCOMO), introduced by Barry Boehm, is used to
estimate the development resources.
COCOMO provides a systematic approach for evaluating software projects based on
size and development characteristics. The three categories of the COCOMO

Plagiarism Report for Chapter 6



Primary Sources

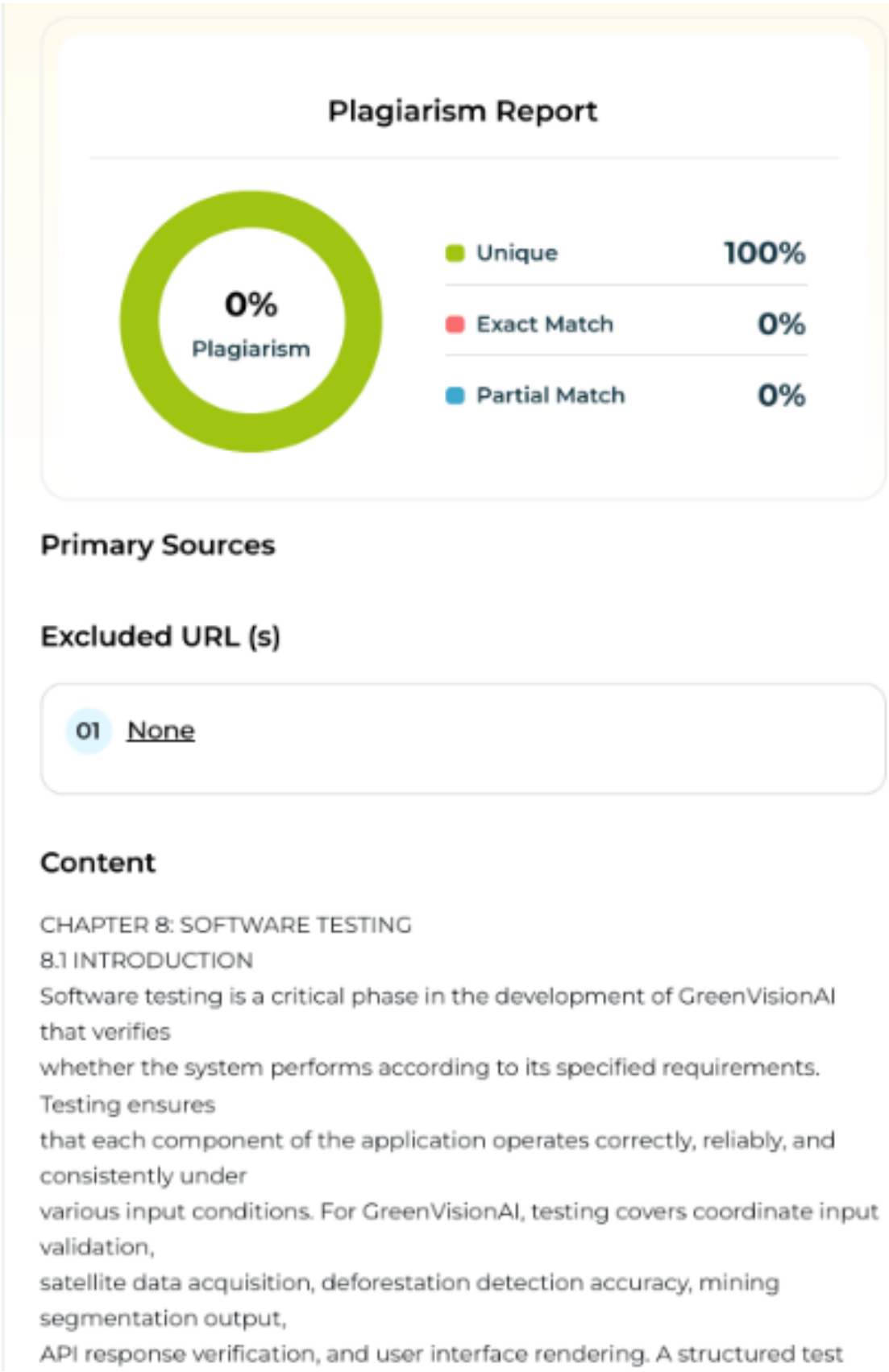
Excluded URL (s)

01 None

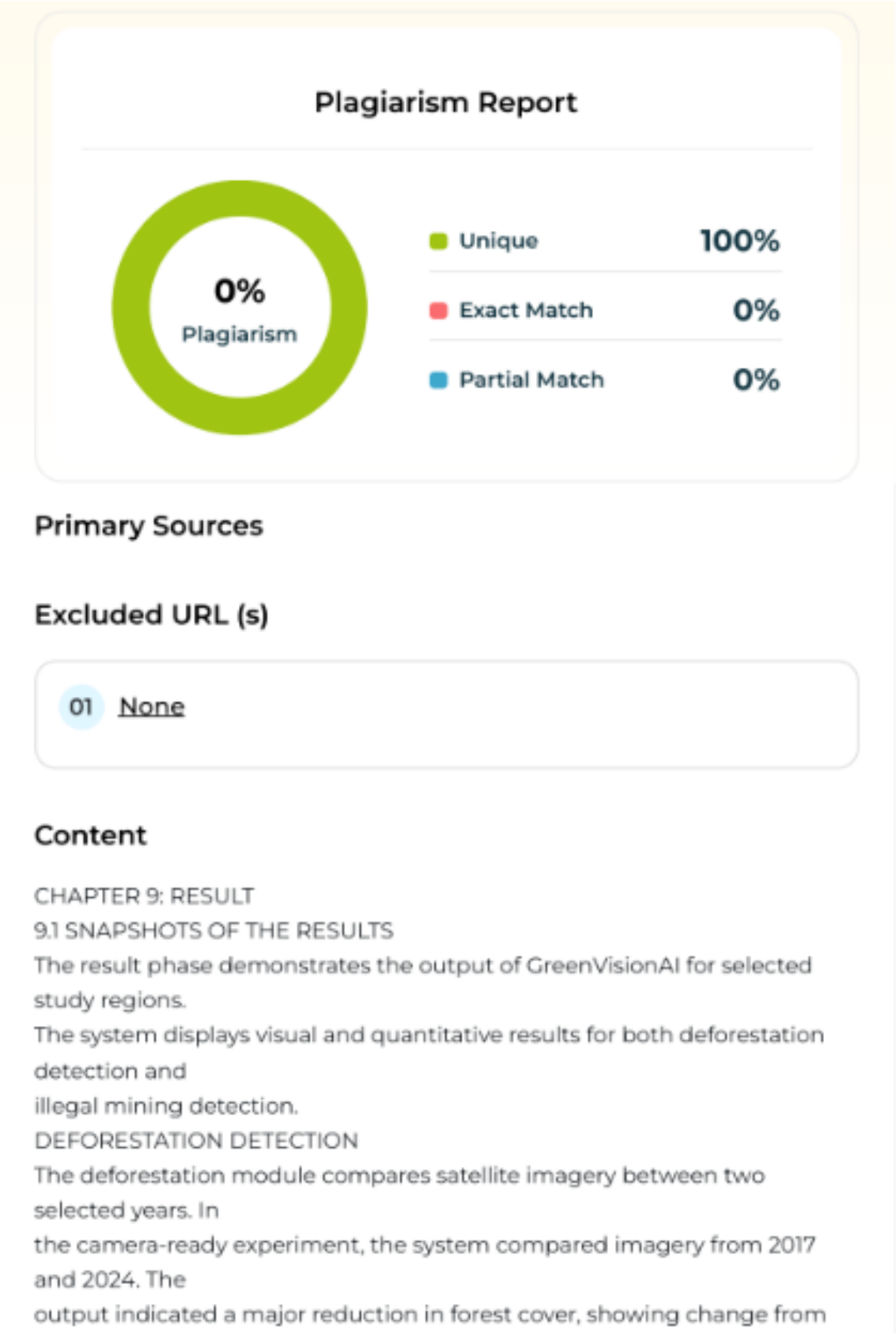
Content

CHAPTER 7: SOFTWARE
IMPLEMENTATION
7.1 INTRODUCTION
Software implementation converts the system design into a working application.
GreenVisionAI is implemented as a modular web application with separate frontend,
backend, and AI processing components. The implementation focuses on two distinct
processing pipelines: a Random Forest classification pipeline for deforestation detection
and a U-Net segmentation pipeline for illegal mining detection, both driven by Sentinel-2

Plagiarism Report for Chapter 7



Plagiarism Report for Chapter 8



Plagiarism Report

0%
Plagiarism

Unique 100%
Exact Match 0%
Partial Match 0%

Primary Sources

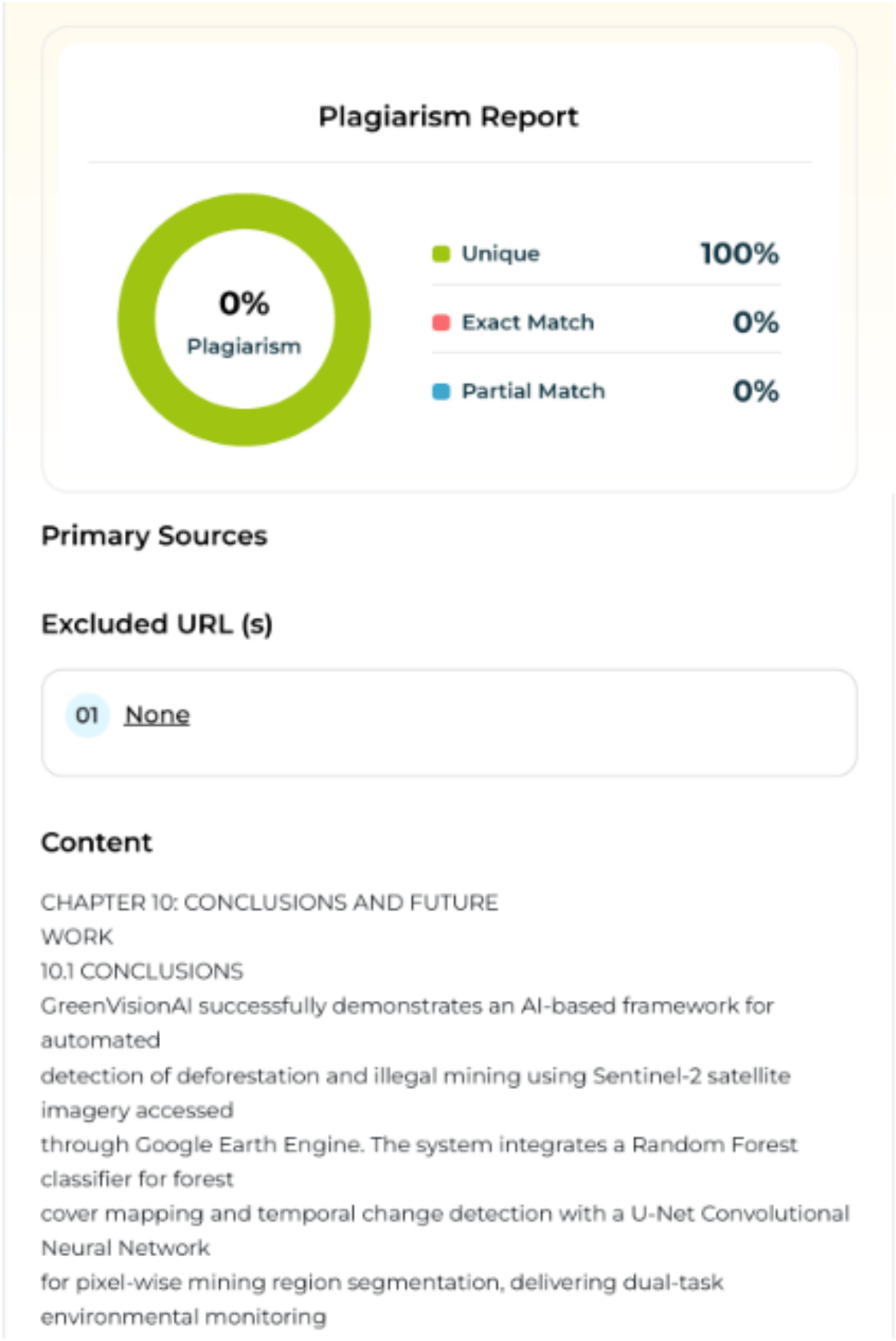
Excluded URL (s)

01 None

Content

CHAPTER 9: RESULT
9.1 SNAPSHOTS OF THE RESULTS
The result phase demonstrates the output of GreenVisionAI for selected study regions.
The system displays visual and quantitative results for both deforestation detection and
illegal mining detection.
DEFORESTATION DETECTION
The deforestation module compares satellite imagery between two selected years. In
the camera-ready experiment, the system compared imagery from 2017 and 2024. The
output indicated a major reduction in forest cover, showing change from

Plagiarism Report for Chapter 9



Plagiarism Report for Chapter 10

ANNEXURE E
PUBLICATIONS AND CERTIFICATES

Attach publication proof, conference certificates, project competition certificates, participation certificates or acceptance letters related to GreenVisionAI in this annexure.

PUBLICATION DETAILS

Publication Details — Conference Paper (ICETES-2026)

Field	**Detail**
Sr. No	1
Name of Authors	Barkare Vedika Nandkishor, Patil Pranjali Mohan, Patil Shubham Dnyaneshwar, Sonawane Pravin Rajesh, Prof. S. P. Baviskar
Title of Paper	GreenVisionAI: An AI-Based Framework for Detecting Illegal Mining and Deforestation Using Sentinel-2 Satellite Imagery
Name of Conference	2nd International Conference on Emerging Trends in Engineering and Sciences (ICETES-2026)
Date of Publication	10–11 April 2026
Organizing Institute	Jawahar Education Society's Institute of Technology, Management and Research (JESITMR), Nashik
ISBN or ISSN	ISSN: 978-93-5636-945-0

CERTIFICATES



ICETES-2026 Certificate — Barkare Vedika Nandkishor



ICETES-2026 Certificate — Patil Pranjali Mohan



ICETES-2026 Certificate — Patil Shubham Dnyaneshwar



ICETES-2026 Certificate — Sonawane Pravin Rajesh